

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и
радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

ОТЧЕТ

по лабораторной работе №3

по дисциплине

**«Современные технологии программирования масштабируемых
приложений»**

«Кредитные операции с физическими лицами»

Выполнил:

Лебедев Артем Владимирович
магистрант группы 556301

Преподаватель:

Хмелёва Ангелина Владимировна
кандидат технических наук
доцент кафедры ПОИТ

Минск, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	1
2	Задание	2
3	Проектирование	3
3.1	Место сервиса кредитов в системе	4
3.2	План счетов.....	4
3.3	Схема данных	5
3.4	Схема проводок	5
3.5	Графики платежей	6
3.6	Заккрытие банковского дня	7
3.7	REST-интерфейс	7
4	Реализация	7
4.1	Структура модуля.....	8
4.2	Расчёт графика платежей	8
4.3	Заключение договора	9
4.4	Контроль корректности данных договора	10
4.5	Обработка банковского дня.....	10
5	Тестирование	11
6	Результаты работы	12
	Заключение	21
	Список использованных источников.....	22
	Приложение А Исходный код сервиса кредитов	23

1 ЦЕЛЬ РАБОТЫ

Реализовать микросервис кредитных операций с физическими лицами: справочник кредитных программ, форму заключения кредитного договора с контролем вводимых данных, открытие счетов клиента по плану счетов, график платежей для двух способов погашения, проводки по схеме «Кредитная программа» с использованием фонда развития банка и участие в процедуре «Закрытие банковского дня» вместе с депозитными программами предыдущей работы.

2 ЗАДАНИЕ

Вариант № 4 — банк «Решение». Выбраны две кредитные программы для физических лиц (таблица 2.1); условия приближены к действующим предложениям банка. Банки Беларуси кредитуют физических лиц в белорусских рублях, поэтому обе программы рублёвые; справочник валют в форме договора при этом используется.

Таблица 2.1 – Кредитные программы банка «Решение»

	«R-деньги mix»	«R-Онлайн»
Способ погашения	ежемесячно равными аннуитетными платежами	ежемесячно проценты, вся сумма кредита в конце срока
Ставка	17,65 % годовых	17,65 % годовых
Сумма кредита	500–50 000 BYN	300–15 000 BYN
Срок договора	6–60 месяцев	3–36 месяцев
Счёт основной суммы	2400	2400
Счёт процентов	2470	2470

Требования задания:

1. внести в справочник «План счетов» счета, необходимые для обслуживания кредитных линий;
2. разработать форму заключения кредитного договора (вид кредита и валюта из справочников, номер договора, даты, срок, сумма, процент) с контролем корректности данных;
3. при заключении договора создавать не менее двух счетов — для основной суммы и для процентов по кредиту;
4. использовать счёт фонда развития банка из предыдущей работы со стартовым капиталом 100 млрд белорусских рублей;
5. сформировать график платежей;
6. разработать процедуру «Закрытие банковского дня» и отчёт о состоянии счетов; при двух активных кредитных программах в нём должно быть не менее десяти счетов с учётом депозитных.

3 ПРОЕКТИРОВАНИЕ

3.1 Место сервиса кредитов в системе

Для кредитных операций добавлен четвёртый микросервис, *credit-service* (рисунок 3.1). Он устроен так же, как сервис депозитов: хранит программы и договоры в собственной базе, клиента запрашивает у модуля «Клиенты», а счета и проводки ведёт через главную книгу. Код существующих сервисов при этом не меняется: главная книга открывает счета любого балансового кода и оповещает о новом банковском дне всех участников из списка, поэтому подключение кредитов сводится к двум строкам справочника «План счетов» и одной записи в настройке списка участников.

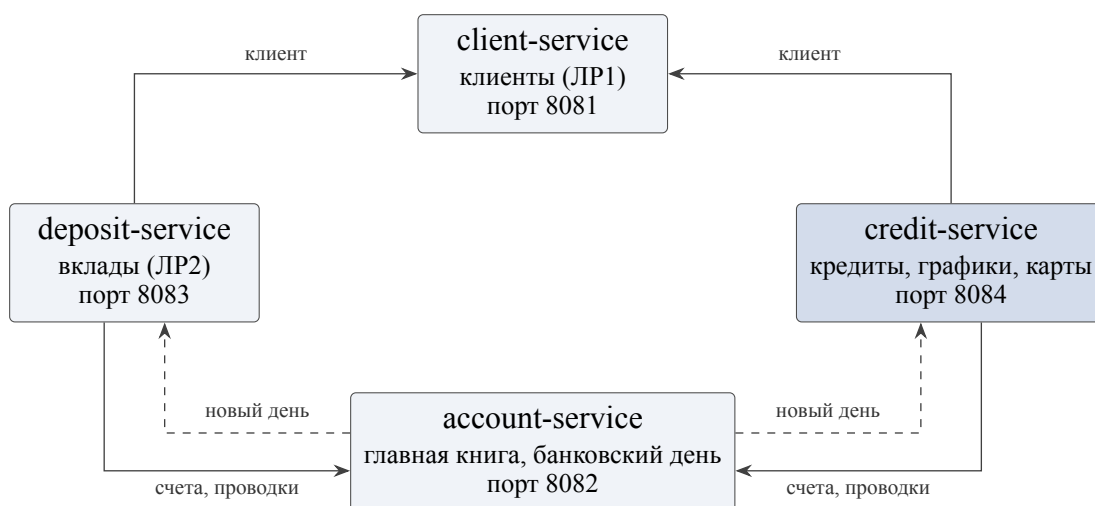


Рисунок 3.1 – Сервис кредитов среди микросервисов банка

3.2 План счетов

Для кредитных программ в план счетов добавлены два активных счёта (таблица 3.1). В отличие от депозитных, оба счёта клиента активные: их остаток растёт по дебету и уменьшается по кредиту.

Таблица 3.1 – Счета плана счетов, используемые кредитными программами

Код	Активность	Наименование	Назначение в работе
1010	активный	Денежные средства в кассе	касса банка
2400	активный	Кредиты физическим лицам	текущий (кредитный) счёт клиента
2470	активный	Начисленные процентные доходы по кредитам физическим лицам	счёт процентов по кредиту
7327	пассивный	Фонд развития банка	источник кредитных средств

3.3 Схема данных

Схема базы сервиса кредитов приведена на рисунке 3.2. График платежей хранится построчно в таблице `payment_item`: это позволяет отмечать оплату каждого платежа и показывать клиенту остаток долга. Таблица `card` относится к следующей работе — к кредитному счёту выпускается карта для банкомата.

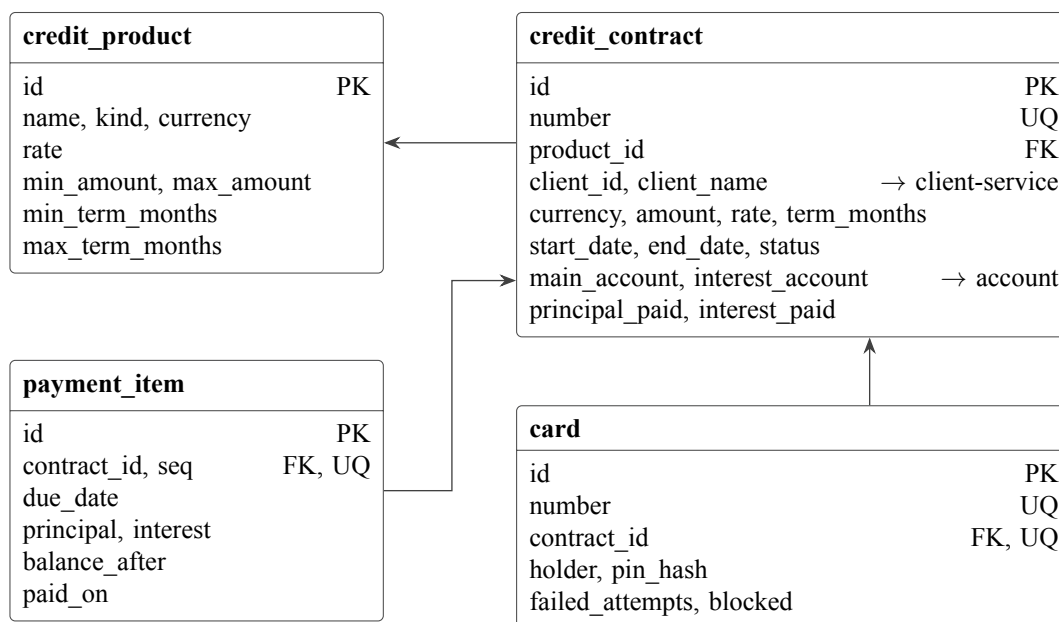


Рисунок 3.2 – Схема данных сервиса кредитов

3.4 Схема проводок

Проводки выполняются по упрощённой схеме «Кредитная программа» из задания (таблица 3.2). Знак «+» означает увеличение остатка счёта, «–» — уменьшение: фонд развития пассивный, остальные три счёта активные.

Таблица 3.2 – Проводки процесса «Кредитная программа»

№	Операция	Касса (А)		СФРБ (П)		Текущий счёт (А)		Проценты (А)	
		Дт	Кт	Дт	Кт	Дт	Кт	Дт	Кт
1	Выделение кредита банком			–		+			
2	Перевод кредита в кассу	+					–		
3	Получение кредита через кассу		–						
4	Внесение процентов в кассу	+							
5	Перевод процентов из кассы		–					+	
6	Начисление процентов банком				+				–
7	Внесение денег в кассу	+							
8	Погашение долга за кредит из кассы		–			+			
9	Окончание кредита				+		–		

Операция 1 выполняется при заключении договора. Операции 2–3

— при получении денег клиентом: в кассе отделения либо в банкомате (лабораторная работа № 4), причём частями. Остаток текущего счёта поэтому показывает, сколько кредитных средств клиент ещё может снять. Операции 4–6 проводятся в дату уплаты процентов, 7–9 — в дату погашения основного долга; при аннуитетном погашении операция 9 возвращает в фонд очередную часть долга, а название «Окончание кредита» носит последняя из них.

В задании операция «Начисление процентов банком» стоит перед внесением процентов в кассу. В реализации проценты сначала вносятся клиентом и только затем списываются банком с процентного счёта: итоговые обороты те же, но остаток счёта ни в какой момент не становится отрицательным, а главная книга отрицательных остатков не допускает.

3.5 Графики платежей

Месячная процентная ставка равна $i = r/(12 \cdot 100)$, где r — годовая ставка.

Аннуитет. Ежемесячный платёж постоянен и равен

$$A = S \cdot \frac{i(1+i)^n}{(1+i)^n - 1}, \quad (3.1)$$

где S — сумма кредита, n — срок в месяцах. В платеже номер k проценты начисляются на остаток долга B_{k-1} , остальное идёт в погашение:

$$I_k = \text{round}(B_{k-1} \cdot i, 2), \quad P_k = A - I_k, \quad B_k = B_{k-1} - P_k. \quad (3.2)$$

Последний платёж закрывает остаток целиком ($P_n = B_{n-1}$) и вбирает накопленную погрешность округления, поэтому сумма погашений всегда в точности равна сумме кредита.

Ежемесячная уплата процентов. Долг не уменьшается до конца срока, поэтому каждый месяц уплачивается одна и та же сумма $I = \text{round}(S \cdot i, 2)$, а последний платёж равен $S + I$.

Для кредита 12 000 BYN на 6 месяцев под 17,65 % годовых $i = 0,0147083$, $A = 2104,21$ BYN; график приведён в таблице 3.3. Тот же кредит с ежемесячной уплатой процентов потребовал бы шести платежей по 176,50 BYN и возврата 12 000 BYN в конце: переплата составила бы 1 059,00 BYN против 625,27 BYN при аннуитете, поскольку долг всё время остаётся полным.

Таблица 3.3 – Аннуитетный график платежей: 12 000 BYN, 6 месяцев, 17,65 % годовых

№	Основной долг	Проценты	Платёж	Остаток долга
1	1 927,71	176,50	2 104,21	10 072,29
2	1 956,06	148,15	2 104,21	8 116,23
3	1 984,83	119,38	2 104,21	6 131,40
4	2 014,03	90,18	2 104,21	4 117,37
5	2 043,65	60,56	2 104,21	2 073,72
6	2 073,72	30,50	2 104,22	0,00
Итого	12 000,00	625,27	12 625,27	

3.6 Заккрытие банковского дня

Процедура закрытия дня осталась прежней: главная книга объявляет участникам дату нового дня и сдвигает её после успешной обработки. Участников теперь два — сервисы депозитов и кредитов. Сервис кредитов в ответ проводит все платежи графиков, срок которых наступил; по условию задания клиент платит без задержки. Ключ пакета проводок включает номер платежа (CRD-K-000001-PAY-3), поэтому повторная обработка дня не проведёт платёж дважды, а после простоя сервис проведёт сразу все пропущенные платежи. Когда оплачена последняя строка графика, договор закрывается.

3.7 REST-интерфейс

Операции сервиса кредитов приведены в таблице 3.4.

Таблица 3.4 – REST-интерфейс сервиса кредитов

Метод	Адрес	Назначение
GET	/api/products, /api/meta	виды кредитов, валюты, банковский день, номер договора
GET	/api/clients	список клиентов из модуля «Клиенты»
POST	/api/credits/schedule	предварительный расчёт графика платежей
POST	/api/credits	заключение договора; в ответе — договор и PIN-конверт карты
GET	/api/credits, /api/credits/{id}	договоры; договор с графиком, счетами и проводками
POST	/api/credits/{id}/cash	выдача наличных с кредитного счёта через кассу
POST	/api/credits/{id}/pin	перевыпуск PIN-кода карты
POST	/api/internal/day-close	обработка нового банковского дня
POST	/api/atm/transactions	транзакции банкомата (лабораторная работа № 4)

4 РЕАЛИЗАЦИЯ

4.1 Структура модуля

Структура модуля `credit-service` показана в листинге 4.1.

Листинг 4.1 – Модуль `credit-service`

domain/	CreditProduct, CreditContract, PaymentItem, CreditKind, Card, CardNumbers
repository/	репозитории Spring Data JPA
dto/	CreditRequest, CreditView, ScheduleRow, ScheduleView, CreditIssued
service/	CreditService заключение договора, выдача, обработка дня ScheduleCalculator расчёт графиков платежей CreditPostings проводки по схеме задания CardService выпуск карты, PIN-код DayCloseProcessor участие в закрытии банковского дня
atm/	банковская сторона протокола банкомата (ЛР4)
web/	CreditController, ReferenceController, DayCloseController, AtmController
resources/	schema.sql, data.sql (программы банка), static/ (web-клиент)

4.2 Расчёт графика платежей

Расчёт вынесен в класс без состояния `ScheduleCalculator` (листинг 4.2) и используется дважды: для предварительного показа графика в форме и при заключении договора. Вычисления ведутся в `BigDecimal`; степень $(1 + i)^n$ считается с точностью 16 значащих цифр, денежные суммы округляются до копеек по правилу «половина вверх».

Листинг 4.2 – Аннуитетный график (`ScheduleCalculator`)

```
/** Аннуитетный платёж:  $A = S * i / (1 - (1 + i)^{-n})$ . */
public static BigDecimal annuityPayment(BigDecimal amount, BigDecimal rate, int months) {
    BigDecimal i = rate.divide(MONTHS_PERCENT, PRECISION);
    if (i.signum() == 0) {
        return amount.divide(BigDecimal.valueOf(months), 2, RoundingMode.HALF_UP);
    }
    BigDecimal growth = BigDecimal.ONE.add(i).pow(months, PRECISION);
    return amount.multiply(i).multiply(growth)
        .divide(growth.subtract(BigDecimal.ONE), 2, RoundingMode.HALF_UP);
}

private static List<ScheduleRow> annuity(BigDecimal amount, BigDecimal rate, int months,
    LocalDate start) {
    BigDecimal payment = annuityPayment(amount, rate, months);
    BigDecimal balance = amount;
    List<ScheduleRow> rows = new ArrayList<>();
    for (int k = 1; k <= months; k++) {
        BigDecimal interest = monthlyInterest(balance, rate);
        BigDecimal principal = k == months ? balance : payment.subtract(interest).min(balance);
        balance = balance.subtract(principal);
        rows.add(new ScheduleRow(k, start.plusMonths(k), principal, interest, balance));
    }
    return rows;
}
```

4.3 Заключение договора

При заключении договора сервис проверяет условия программы, открывает в главной книге счета 2400 и 2470 на имя клиента, выделяет кредит из фонда развития, сохраняет договор с графиком и выпускает карту (листинг 4.3). Если в фонде недостаточно средств, главная книга отклоняет проводку, и её сообщение доходит до формы.

Листинг 4.3 – Счета, выделение кредита, график и карта
(CreditService.open)

```
String main = ledger.open(new OpenAccount(MAIN_CHART_CODE, currency,
    client.id().intValue(), client.fullName(), number)).number();
String interest = ledger.open(new OpenAccount(INTEREST_CHART_CODE, currency,
    client.id().intValue(), client.fullName(), number)).number();
post(key(number, "ISSUE"), number, today, List.of(CreditPostings.issue(fund(currency), main,
    request.amount())));

CreditContract contract = new CreditContract();
...
for (ScheduleRow row : ScheduleCalculator.build(product.getKind(), request.amount(), product.
    getRate(),
    request.termMonths(), today)) {
    PaymentItem item = new PaymentItem();
    ...
    contract.getSchedule().add(item);
}
contracts.saveAndFlush(contract);
PinEnvelope card = cardService.issue(contract);
return new CreditIssued(view(contract), card);
```

Проводки кредитной программы собраны в классе CreditPostings (листинг 4.4); платёж по графику формирует до шести операций схемы, которые главная книга проводит одним атомарным пакетом.

Листинг 4.4 – Проводки выдачи кредита и платежа по графику
(CreditPostings)

```
public static Posting issue(String fund, String main, BigDecimal amount) {
    return Posting.of("Выделение кредита банком", Entry.debit(fund, amount), Entry.debit(main,
        amount));
}

public static List<Posting> payment(String cash, String fund, String main, String
    interestAccount,
    BigDecimal interest, BigDecimal principal, boolean last) {
    List<Posting> postings = new ArrayList<>();
    if (interest.signum() > 0) {
        postings.add(Posting.of("Внесение процентов в кассу", Entry.debit(cash, interest)));
        postings.add(Posting.of("Перевод процентов из кассы", Entry.credit(cash, interest),
            Entry.debit(interestAccount, interest)));
        postings.add(Posting.of("Начисление процентов банком", Entry.credit(interestAccount,
            interest), Entry.credit(fund, interest)));
    }
    if (principal.signum() > 0) {
        postings.add(Posting.of("Внесение денег в кассу", Entry.debit(cash, principal)));
        postings.add(Posting.of("Погашение долга за кредит из кассы", Entry.credit(cash,
```

```

principal), Entry.debit(main, principal)));
    postings.add(Posting.of(last ? "Окончание кредита" : "Возврат части кредита в фонд
банка",
        Entry.credit(main, principal), Entry.credit(fund, principal)));
}
return postings;
}

```

4.4 Контроль корректности данных договора

Форма договора проверяется дважды: в web-клиенте до отправки и в сервисе (таблица 4.1).

Таблица 4.1 – Проверки формы кредитного договора

Поле	Правило	Где проверяется
Клиент	выбран; существует в модуле «Клиенты»	клиент, сервис
Номер договора	формат К-000001; не занят	клиент, сервис
Вид кредита	запись справочника; доступен в выбранной валюте	клиент, сервис
Сумма	денежная; в пределах минимума и максимума программы	клиент, сервис
Срок	целое число месяцев в пределах программы	клиент, сервис
Процент	равен ставке программы	сервис
Дата начала	равна текущему банковскому дню	сервис
Дата окончания	равна дате начала плюс срок	сервис
Средства банка	суммы достаточно в фонде развития	главная книга

4.5 Обработка банковского дня

При открытии нового дня сервис проходит по графику каждого действующего договора и проводит неоплаченные платежи, срок которых наступил (листинг 4.5).

Листинг 4.5 – Платежи по графику (CreditService.processDay)

```

for (PaymentItem item : contract.getSchedule()) {
    if (item.isPaid() || item.getDueDate().isAfter(date)) {
        continue;
    }
    boolean last = item.getSeq() == contract.getTermMonths();
    post(key(number, "PAY-" + item.getSeq()), number, date, CreditPostings.payment(cash(
        currency), fund(currency),
        contract.getMainAccount(), contract.getInterestAccount(), item.getInterest(), item
        .getPrincipal(), last));
    item.setPaidOn(date);
    contract.setInterestPaid(contract.getInterestPaid().add(item.getInterest()));
    contract.setPrincipalPaid(contract.getPrincipalPaid().add(item.getPrincipal()));
}
if (contract.getSchedule().stream().allMatch(PaymentItem::isPaid)) {
    contract.setStatus(ContractStatus.CLOSED);
    contract.setClosedOn(date);
}

```

5 ТЕСТИРОВАНИЕ

Сервис кредитов покрыт 73 тестами (таблица 5.1). Как и в предыдущей работе, главная книга в тестах заменена двойником `InMemoryLedger` из тестовой части общей библиотеки, а REST-клиенты модулей «Клиенты» и «Депозиты» — заглушками `Mockito`.

Таблица 5.1 – Тесты сервиса кредитов

Класс тестов	Тестов	Что проверяется
<code>ScheduleCalculatorTest</code>	8	формула аннуитета, оба графика, округление
<code>CardNumbersTest</code>	10	номер карты, счётчик попыток PIN-кода
<code>CreditFlowIT</code>	7	проводки по схеме, платежи, закрытие договора
<code>CreditApiIT</code>	23	контроль данных договора, график, выдача через кассу
<code>AtmProtocolIT</code>	25	протокол банкомата (лабораторная работа № 4)
Всего	73	

Тест первого платежа по аннуитетному кредиту сверяет обороты всех четырёх счетов с таблицей 3.2 (листинг 5.1); итог прогона приведён в листинге 5.2.

Листинг 5.1 – Тест платежа по графику (`CreditFlowIT`)

```
@Test
@DisplayName("Аннуитет: в дату платежа проводятся проценты и часть долга, остальные дни счета не меняются")
void annuityPaymentOnDueDate() {
    CreditView credit = open("K-000001", ANNUITY, "10000.00", 6).contract();
    credits.cashOut(credit.id(), new BigDecimal("10000.00"));

    assertThat(closeDays(30)).isEmpty(); // 31.10 – платежей ещё нет
    List<String> events = closeDays(1); // 01.11 – первый платёж
    assertThat(events).containsExactly("01.11.2026: K-000001 – платёж №1: проценты 147.08, основной долг 1606.43 BYN");

    // процентный счёт: дебет +147,08 (перевод из кассы) и кредит -147,08 (начисление банком)
    assertTurnover(credit.interestAccount(), "147.08", "147.08", "0.00");
    // текущий счёт: +10000 выдача, -10000 снятие, +1606,43 погашение из кассы, -1606,43 возврат в фонд
    assertTurnover(credit.mainAccount(), "11606.43", "11606.43", "0.00");
    // касса: транзит выданного кредита и принятого платежа 1753,51
    assertTurnover(cash, "11753.51", "11753.51", "0.00");
    // фонд: -10000 выдача, +147,08 проценты, +1606,43 возврат долга
    assertThat(ledger.balance(fund)).isEqualByComparingTo(CAPITAL.subtract(new BigDecimal("8246.49")));
}
```

Листинг 5.2 – Результат выполнения тестов

```
$ ./mvnw -pl credit-service -am verify
```

```
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.055 s -- in by.bsuir.bank.credit.CardNumbersTest
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in by.bsuir.bank.credit.ScheduleCalculatorTest
Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.681 s -- in by.bsuir.bank.credit.CreditFlowIT
Tests run: 25, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.605 s -- in by.bsuir.bank.credit.AtmProtocolIT
Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.301 s -- in by.bsuir.bank.credit.CreditApiIT
```

```
Tests run: 73, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

6 РЕЗУЛЬТАТЫ РАБОТЫ

Сценарий демонстрации: в банке действуют два вклада из предыдущей работы; два клиента берут кредиты по разным программам; банк работает месяц. Фонд развития запущен со стартовым капиталом 100 млрд BYN.

Контроль данных формы показан на рисунке 6.1: сумма и срок выходят за пределы программы «R-Онлайн». На рисунке 6.2 — корректно заполненная форма с предварительным расчётом графика: кнопка «Рассчитать график» запрашивает расчёт у сервиса, не создавая договор. Числа графика совпадают с таблицей 3.3.

Банк «Решение» учебная АБС Клиенты Счета Депозиты **Кредиты** Банкомат

Кредитные договоры
Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.10.2026 **Закреть день** 30 **Закреть дней**

Новый кредитный договор

Форма заполнена с ошибками — исправьте отмеченные поля

Клиент *
Лебедев Артем Владимирович (паспорт МР 3141592)

Номер договора *
K-000001

Валюта *
BYN — Белорусский рубль

Вид кредита *
«R-Онлайн» — проценты ежемесячно, долг в конце срока, 17.65 % годовых
Кредит с ежемесячной уплатой процентов и возвратом всей суммы в конце срока. Сумма 300,00–15 000,00 BYN, срок 3–36 мес.

Сумма кредита, BYN *
20000
Сумма кредита по программе: от 300,00 до 15 000,00 BYN

Срок договора, мес. *
48
Срок по программе: от 3 до 36 мес.

Процент по кредиту, % годовых
17.65

Дата начала
01.10.2026

Дата окончания
01.10.2030

ЗаклЮчить договор Рассчитать график Отмена

Рисунок 6.1 – Контроль корректности данных кредитного договора

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Кредитные договоры

Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Заккрыть день

30

Заккрыть дней

Новый кредитный договор

Клиент *

Лебедевич Артем Владимирович (паспорт МР 3141592)

Номер договора *

K-000001

Валюта *

BYN — Белорусский рубль

Вид кредита *

«R-деньги mix» — аннуитетные платежи, 17.65 % годовых

Сумма кредита, BYN *

12000

Срок договора, мес. *

6

Процент по кредиту, % годовых

17.65

Дата начала

01.10.2026

Дата окончания

01.04.2027

Заклчить договор

Рассчитать график

Отмена

График платежей (предварительный расчёт)

№	ДАТА ПЛАТЕЖА	ОСНОВНОЙ ДОЛГ	ПРОЦЕНТЫ	ПЛАТЁЖ	ОСТАТОК ДОЛГА	ОПЛАЧЕН
1	01.11.2026	1 927,71	176,50	2 104,21	10 072,29	
2	01.12.2026	1 956,06	148,15	2 104,21	8 116,23	
3	01.01.2027	1 984,83	119,38	2 104,21	6 131,40	
4	01.02.2027	2 014,03	90,18	2 104,21	4 117,37	
5	01.03.2027	2 043,65	60,56	2 104,21	2 073,72	
6	01.04.2027	2 073,72	30,50	2 104,22	0,00	
Итого, BYN			625,27	12 625,27		

Рисунок 6.2 – Форма кредитного договора с графиком платежей

После заключения договора открывается его карточка (рисунок 6.3). Кредит зачислен на текущий счёт 2400, к счёту выпущена карта; номер карты и PIN-код показаны в «ПИН-конверте» один раз — в базе хранится только хеш PIN-кода.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Кредитные договоры

Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Заккрыть день

30

Заккрыть дней

Договор K-000001 заключён, кредит зачислен на счёт 2400000010030

ПИН-конверт. Карта 9112 3800 0000 0010, PIN-код 5093. PIN-код показывается один раз и в базе не хранится — передайте его клиенту.

Договор K-000001 · Лебедевич Артем Владимирович

К списку

ВИД КРЕДИТА

«Р-деньги mix»

Аннуитетные платежи

СУММА КРЕДИТА

12 000,00 BYN

СТАВКА

17.65 % годовых

СТАТУС

Действует

СРОК

6 мес.

ПЕРИОД

01.10.2026 — 01.04.2027

ОСТАТОК ДОЛГА

12 000,00 BYN

УПЛАЧЕНО ПРОЦЕНТОВ

0,00 BYN

Карта и выдача наличных

КАРТА К КРЕДИТНОМУ СЧЁТУ

9112 3800 0000 0010

Активна

ДОСТУПНО НА СЧЁТЕ

12 000,00 BYN

Выдать через кассу, BYN

0.00

Выдать

Перевыпустить PIN

График платежей

№	ДАТА ПЛАТЕЖА	ОСНОВНОЙ ДОЛГ	ПРОЦЕНТЫ	ПЛАТЁЖ	ОСТАТОК ДОЛГА	ОПЛАЧЕН
1	01.11.2026	1 927,71	176,50	2 104,21	10 072,29	
2	01.12.2026	1 956,06	148,15	2 104,21	8 116,23	
3	01.01.2027	1 984,83	119,38	2 104,21	6 131,40	
4	01.02.2027	2 014,03	90,18	2 104,21	4 117,37	
5	01.03.2027	2 043,65	60,56	2 104,21	2 073,72	
6	01.04.2027	2 073,72	30,50	2 104,22	0,00	
Итого, BYN			625,27	12 625,27		

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛЬДО
2400000010030	2400	Кредиты физическим лицам Лебедевич Артем Владимирович	Активный	12 000,00	0,00	12 000,00
2470000010042	2470	Начисленные процентные доходы по кредитам физическим лицам Лебедевич Артем Владимирович	Активный	0,00	0,00	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
01.10.2026	Выделение кредита банком	73270000000029 7327	12 000,00	
		2400000010030 2400		12 000,00

Рисунок 6.3 – Карточка кредитного договора после заключения

Первый клиент получил всю сумму через кассу, второй оставил деньги на счёте. Список договоров приведён на рисунке 6.4.

15

Банк «Решение» учебная АБС Клиенты Счета Депозиты Кредиты Банкомат									
Кредитные договоры Кредиты физических лиц: заключение договора, график платежей, погашение БАНКОВСКИЙ ДЕНЬ 01.10.2026 Заккрыть день 30 Заккрыть дней									
Договоров: 2 Закключить договор									
ДОГОВОР	КЛИЕНТ	ВИД КРЕДИТА	СУММА	СТАВКА	НАЧАЛО	ОКОНЧАНИЕ	ОСТАТОК ДОЛГА	УПЛАЧЕНО ПРОЦЕНТОВ	СТАТУС
K-000002	Жуков Дмитрий Александрович	Р-Онлайн Проценты ежемесячно, долг в конце срока	6 000,00 BYN	17.65 %	01.10.2026	01.02.2027	6 000,00	0,00	Действует
K-000001	Лебедевич Артем Владимирович	Р-деньги mix Аннуитетные платежи	12 000,00 BYN	17.65 %	01.10.2026	01.04.2027	12 000,00	0,00	Действует

Рисунок 6.4 – Список кредитных договоров

В отчёте по счетам (рисунок 6.5) десять счетов в белорусских рублях: касса, фонд развития, по два счёта на каждый из двух кредитных и двух депозитных договоров. Фонд развития равен $100\,000\,000\,000 + 15\,000 - 18\,000 = 99\,999\,997\,000$ BYN: он вырос на сумму вкладов и уменьшился на сумму выданных кредитов. На текущем счёте второго клиента остаётся 6 000 BYN, у первого — ноль: дебетовый оборот 12 000 (выдача) равен кредитовому (снятие через кассу).

Банк «Решение»

учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.10.2026

Закрыть день

30

Закрыть дней

ОТЧЁТ НА ДАТУ

01.10.2026

СЧЕТОВ В ОТЧЁТЕ

10

ФОНД РАЗВИТИЯ, BYN

99 999 997 000,00

КАССА, BYN

0,00

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛДОО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		27 000,00	27 000,00	0,00
2400000010030	2400	Лебедев Артем Владимирович Кредиты физическим лицам	Активный	К-000001	12 000,00	12 000,00	0,00
2400000050018	2400	Жуков Дмитрий Александрович Кредиты физическим лицам	Активный	К-000002	6 000,00	0,00	6 000,00
2470000010042	2470	Лебедев Артем Владимирович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000001	0,00	0,00	0,00
2470000050020	2470	Жуков Дмитрий Александрович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000002	0,00	0,00	0,00
3404000010017	3404	Лебедев Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	10 000,00	10 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	5 000,00	5 000,00	0,00
3470000010025	3470	Лебедев Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	0,00	0,00	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	0,00	0,00	0,00
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		18 000,00	100 000 015 000,00	99 999 997 000,00
Итого обороты, BYN					78 000,00	100 000 069 000,00	

Рисунок 6.5 – Отчёт по счетам: две кредитные и две депозитные программы

После 31 закрытия дня наступает дата первых платежей (рисунок 6.6). По аннуитетному кредиту уплачено 176,50 BYN процентов и 1 927,71 BYN основного долга, по кредиту «R-Онлайн» — только проценты $6000 \cdot 17,65/1200 = 88,25$ BYN. В том же протоколе видны операции сервиса депозитов: оба участника обработали один и тот же день.

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Счета банка

Отчёт о состоянии счетов и процедура «Закрытие банковского дня»

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Закрыть день

31

Закрыть дней

ОТЧЁТ НА ДАТУ

01.11.2026

СЧЕТОВ В ОТЧЁТЕ

10

ФОНД РАЗВИТИЯ, BYN

99 999 999 078,23

КАССА, BYN

0,00

Протокол закрытия дня

28.10.2026: Д-000002 — начислены проценты 1.76 BYN
29.10.2026: Д-000001 — начислены проценты 1.92 BYN
29.10.2026: Д-000002 — начислены проценты 1.77 BYN
30.10.2026: Д-000001 — начислены проценты 1.92 BYN
30.10.2026: Д-000002 — начислены проценты 1.77 BYN
31.10.2026: Д-000001 — начислены проценты 1.91 BYN
31.10.2026: Д-000002 — начислены проценты 1.76 BYN
01.11.2026: Д-000001 — начислены проценты 1.92 BYN
01.11.2026: Д-000001 — выплачены проценты 59.45 BYN
01.11.2026: Д-000002 — начислены проценты 1.77 BYN
01.11.2026: К-000001 — платёж №1: проценты 176.50, основной долг 1927.71 BYN
01.11.2026: К-000002 — платёж №1: проценты 88.25, основной долг 0.00 BYN
... всего записей: 65

Отчёт по счетам

Журнал проводок

План счетов

BYN

USD

EUR

НОМЕР СЧЁТА	КОД	НАЗВАНИЕ СЧЁТА	АКТИВНОСТЬ	ДОГОВОР	ДЕБЕТ	КРЕДИТ	САЛЬДО
1010000000019	1010	Касса банка, BYN Денежные средства в кассе	Активный		29 251,91	29 251,91	0,00
2400000010030	2400	Лебедев Артем Владимирович Кредиты физическим лицам	Активный	К-000001	13 927,71	13 927,71	0,00
2400000050018	2400	Жуков Дмитрий Александрович Кредиты физическим лицам	Активный	К-000002	6 000,00	0,00	6 000,00
2470000010042	2470	Лебедев Артем Владимирович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000001	176,50	176,50	0,00
2470000050020	2470	Жуков Дмитрий Александрович Начисленные процентные доходы по кредитам физическим лицам	Активный	К-000002	88,25	88,25	0,00
3404000010017	3404	Лебедев Артем Владимирович Вклады (депозиты) до востребования физических лиц	Пассивный	Д-000001	10 000,00	10 000,00	0,00
3414000020015	3414	Савицкая Марина Олеговна Срочные вклады (депозиты) физических лиц	Пассивный	Д-000002	5 000,00	5 000,00	0,00
3470000010025	3470	Лебедев Артем Владимирович Начисленные процентные расходы по вкладам (депозитам) до востребования	Пассивный	Д-000001	59,45	59,45	0,00
3471000020025	3471	Савицкая Марина Олеговна Начисленные процентные расходы по срочным вкладам (депозитам)	Пассивный	Д-000002	0,00	54,78	54,78
7327000000029	7327	Фонд развития банка, BYN Фонд развития банка	Пассивный		18 114,23	100 000 017 192,46	99 999 999 078,23
Итого обороты, BYN					82 618,05	100 000 075 751,06	

Рисунок 6.6 – Отчёт по счетам и протокол закрытия дня через месяц

Карточка аннуитетного кредита после первого платежа показана на рисунке 6.7: первая строка графика отмечена оплаченной, остаток долга уменьшился до 10 072,29 BYN, в проводках видны все шесть операций платежа. В карточке кредита «R-Онлайн» (рисунок 6.8) основной долг не изменился.

18

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Кредитные договоры

Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Заккрыть день

30

Заккрыть дней

Договор К-000001 · Лебедевич Артем Владимирович

К списку

ВИД КРЕДИТА
«Р-деньги mix»
Аннуитетные платежи

СУММА КРЕДИТА
12 000,00 BYN

СТАВКА
17.65 % годовых

СТАТУС
Действует

СРОК
6 мес.

ПЕРИОД
01.10.2026 — 01.04.2027

ОСТАТОК ДОЛГА
10 072,29 BYN

УПЛАЧЕНО ПРОЦЕНТОВ
176,50 BYN

Карта и выдача наличных

КАРТА К КРЕДИТНОМУ СЧЁТУ
9112 3800 0000 0010
Активна

ДОСТУПНО НА СЧЁТЕ
0,00 BYN

Выдать через кассу, BYN
0.00

Выдать

Перевыпустить PIN

График платежей

№	ДАТА ПЛАТЕЖА	ОСНОВНОЙ ДОЛГ	ПРОЦЕНТЫ	ПЛАТЁЖ	ОСТАТОК ДОЛГА	ОПЛАЧЕН
1	01.11.2026	1 927,71	176,50	2 104,21	10 072,29	01.11.2026
2	01.12.2026	1 956,06	148,15	2 104,21	8 116,23	
3	01.01.2027	1 984,83	119,38	2 104,21	6 131,40	
4	01.02.2027	2 014,03	90,18	2 104,21	4 117,37	
5	01.03.2027	2 043,65	60,56	2 104,21	2 073,72	
6	01.04.2027	2 073,72	30,50	2 104,22	0,00	
Итого, BYN			625,27	12 625,27		

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛЬДО
2400000010030	2400	Кредиты физическим лицам Лебедевич Артем Владимирович	Активный	13 927,71	13 927,71	0,00
2470000010042	2470	Начисленные процентные доходы по кредитам физическим лицам Лебедевич Артем Владимирович	Активный	176,50	176,50	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
01.11.2026	Возврат части кредита в фонд банка	2400000010030 2400		1 927,71
		7327000000029 7327		1 927,71
01.11.2026	Погашение долга за кредит из кассы	1010000000019 1010		1 927,71
		2400000010030 2400	1 927,71	
01.11.2026	Внесение денег в кассу	1010000000019 1010	1 927,71	
01.11.2026	Начисление процентов банком	2470000010042 2470		176,50
		7327000000029 7327		176,50
01.11.2026	Перевод процентов из кассы	1010000000019 1010		176,50
		2470000010042 2470	176,50	
01.11.2026	Внесение процентов в кассу	1010000000019 1010	176,50	
01.10.2026	Получение кредита через кассу	1010000000019 1010		12 000,00
01.10.2026	Перевод кредита в кассу	2400000010030 2400		12 000,00

Рисунок 6.7 – Аннуитетный кредит после первого платежа

Банк «Решение» учебная АБС

Клиенты

Счета

Депозиты

Кредиты

Банкомат

Кредитные договоры

Кредиты физических лиц: заключение договора, график платежей, погашение

БАНКОВСКИЙ ДЕНЬ 01.11.2026

Заккрыть день

30

Заккрыть дней

Договор К-000002 · Жуков Дмитрий Александрович

К списку

ВИД КРЕДИТА
«Р-Онлайн»
Проценты ежемесячно, долг в конце срока

СУММА КРЕДИТА
6 000,00 BYN

СТАВКА
17.65 % годовых

СТАТУС
Действует

СРОК
4 мес.

ПЕРИОД
01.10.2026 — 01.02.2027

ОСТАТОК ДОЛГА
6 000,00 BYN

УПЛАЧЕНО ПРОЦЕНТОВ
88,25 BYN

Карта и выдача наличных

КАРТА К КРЕДИТНОМУ СЧЁТУ
9112 3800 0000 0028
Активна

ДОСТУПНО НА СЧЁТЕ
6 000,00 BYN

Выдать через кассу, BYN
0.00

Выдать

Перевыпустить PIN

График платежей

№	ДАТА ПЛАТЕЖА	ОСНОВНОЙ ДОЛГ	ПРОЦЕНТЫ	ПЛАТЁЖ	ОСТАТОК ДОЛГА	ОПЛАЧЕН
1	01.11.2026	0,00	88,25	88,25	6 000,00	01.11.2026
2	01.12.2026	0,00	88,25	88,25	6 000,00	
3	01.01.2027	0,00	88,25	88,25	6 000,00	
4	01.02.2027	6 000,00	88,25	6 088,25	0,00	
Итого, BYN			353,00	6 353,00		

Счета договора

НОМЕР СЧЁТА	КОД	НАЗНАЧЕНИЕ	АКТИВНОСТЬ	ДЕБЕТ	КРЕДИТ	САЛЬДО
2400000050018	2400	Кредиты физическим лицам Жуков Дмитрий Александрович	Активный	6 000,00	0,00	6 000,00
2470000050020	2470	Начисленные процентные доходы по кредитам физическим лицам Жуков Дмитрий Александрович	Активный	88,25	88,25	0,00

Проводки по договору

ДАТА	ОПЕРАЦИЯ	СЧЁТ	ДЕБЕТ	КРЕДИТ
01.11.2026	Начисление процентов банком	2470000050020 2470		88,25
		7327000000029 7327		88,25
01.11.2026	Перевод процентов из кассы	1010000000019 1010		88,25
		2470000050020 2470	88,25	
01.11.2026	Внесение процентов в кассу	1010000000019 1010	88,25	
01.10.2026	Выделение кредита банком	7327000000029 7327	6 000,00	
		2400000050018 2400	6 000,00	

Рисунок 6.8 – Кредит с ежемесячной уплатой процентов после первого платежа

ЗАКЛЮЧЕНИЕ

В работе реализован микросервис кредитных операций с двумя программами банка «Решение»: кредитом с аннуитетным погашением и кредитом с ежемесячной уплатой процентов и возвратом суммы в конце срока. При заключении договора сервис проверяет условия программы, открывает по плану счетов текущий и процентный счета клиента, выделяет кредит из фонда развития банка и формирует график платежей; платежи проводятся в дату их наступления при закрытии банковского дня проводками по схеме из задания.

Работа показала, что принятое ранее разделение на сервисы оправдано: новая предметная область добавлена отдельным микросервисом без изменения кода клиентов, депозитов и главной книги. Общие правила учёта — расчёт сальдо, запрет отрицательного остатка, атомарность и идемпотентность проводок — реализованы один раз в главной книге и одинаково действуют для депозитов и кредитов, а отчёт о состоянии счетов объединяет счета обеих программ.

Корректность расчётов и проводок подтверждена 73 автоматическими тестами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Об установлении Плана счетов бухгалтерского учёта в банках и небанковских кредитно-финансовых организациях Республики Беларусь: постановление Правления Национального банка Республики Беларусь от 29.08.2013 № 506 [Электронный ресурс]. — Режим доступа: <https://www.nbrb.by/legislation/accountingreporting>. — Дата доступа: 01.10.2026.
2. Кредиты на потребительские нужды / ЗАО «Банк «Решение» [Электронный ресурс]. — Режим доступа: <https://rbank.by/life/credits/>. — Дата доступа: 01.10.2026.
3. Четыркин, Е. М. Финансовая математика: учебник / Е. М. Четыркин. — 10-е изд. — М.: Дело, 2011. — 392 с.
4. Ньюмен, С. Создание микросервисов / С. Ньюмен. — 2-е изд. — СПб.: Питер, 2023. — 624 с.
5. Spring Data JPA Reference Documentation [Электронный ресурс]. — Режим доступа: <https://docs.spring.io/spring-data/jpa/reference/>. — Дата доступа: 01.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код сервиса кредитов

Микросервис «Кредиты» (credit-service)

Листинг А.1 – credit-service/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>by.bsuir.bank</groupId>
    <artifactId>bank</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>credit-service</artifactId>
  <name>credit-service</name>
  <description>ЛР3. Микросервис кредитных операций с физическими лицами и процессинг карт для банкомата (ЛР4)</description>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>ui-kit</artifactId>
    </dependency>
    <!-- BCrypt для хранения PIN-кодов карт -->
    <dependency>
      <groupId>org.springframework.security</groupId>
      <artifactId>spring-security-crypto</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>by.bsuir.bank</groupId>
      <artifactId>bank-common</artifactId>
      <version>${project.version}</version>
      <type>test-jar</type>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

```

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-failsafe-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Листинг А.2 – credit-service/src/main/resources/application.yml

```

server:
  port: ${CREDIT_SERVICE_PORT:8084}

spring:
  application:
    name: credit-service
  datasource:
    url: jdbc:h2:mem:credits;DB_CLOSE_DELAY=-1
    username: sa
  sql:
    init:
      mode: always
  jpa:
    hibernate:
      ddl-auto: validate
      open-in-view: false
  h2:
    console:
      enabled: true
  http:
    client:
      connect-timeout: 2s
      read-timeout: 120s

bank:
  services:
    clients: ${CLIENT_SERVICE_URL:http://localhost:8081}
    accounts: ${ACCOUNT_SERVICE_URL:http://localhost:8082}
    deposits: ${DEPOSIT_SERVICE_URL:http://localhost:8083}

```

Листинг А.3 – credit-service/src/main/resources/schema.sql

```

-- Схема данных сервиса кредитов

-- Справочник кредитных программ («вид кредита»)
create table credit_product (
  id          bigint generated by default as identity primary key,
  name        varchar(100)  not null,
  kind        varchar(14)   not null check (kind in ('ANNUITY', 'INTEREST_ONLY')),
  currency    varchar(3)    not null,
  rate        decimal(6, 2) not null,
  min_amount  decimal(19, 2) not null,
  max_amount  decimal(19, 2) not null,
  min_term_months int      not null,
  max_term_months int      not null,
  description  varchar(300) not null
);

create table credit_contract (
  id          bigint generated by default as identity primary key,
  number      varchar(20)   not null unique,
  product_id  bigint       not null references credit_product,
  client_id   bigint       not null,
  client_name varchar(200)  not null,
  currency    varchar(3)    not null,
  amount      decimal(19, 2) not null,
  rate        decimal(6, 2) not null,
  term_months int          not null,
  start_date  date         not null,
  end_date    date         not null,
  main_account varchar(13) not null,
  -- текущий (кредитный) счёт клиента

```

```

interest_account varchar(13) not null, -- счёт процентов по кредиту
status           varchar(8) not null check (status in ('ACTIVE', 'CLOSED')),
principal_paid   decimal(19, 2) not null default 0,
interest_paid    decimal(19, 2) not null default 0,
closed_on        date
);

-- График платежей по кредиту
create table payment_item (
    id          bigint generated by default as identity primary key,
    contract_id bigint not null references credit_contract,
    seq         int not null,
    due_date    date not null,
    principal   decimal(19, 2) not null,
    interest    decimal(19, 2) not null,
    balance_after decimal(19, 2) not null, -- остаток долга после платежа
    paid_on     date,
    constraint uq_payment_item unique (contract_id, seq)
);

-- Банковская карта к кредитному счёту (ЛР4). PIN-код хранится только в виде BCrypt-хеша
create table card (
    id          bigint generated by default as identity primary key,
    number       varchar(16) not null unique,
    contract_id bigint not null unique references credit_contract,
    holder       varchar(200) not null,
    pin_hash     varchar(80) not null,
    failed_attempts int not null default 0,
    blocked      boolean not null default false
);

-- Справочник операторов мобильной связи
create table mobile_operator (
    code varchar(10) primary key,
    name varchar(60) not null
);

-- Платежи за услуги связи, проведённые через банкомат
create table mobile_payment (
    id          bigint generated by default as identity primary key,
    card_number varchar(16) not null,
    operator     varchar(10) not null references mobile_operator,
    phone        varchar(10) not null,
    amount       decimal(19, 2) not null,
    bank_date    date not null,
    reference    varchar(40) not null unique
);

```

Листинг A.4 – credit-service/src/main/resources/data.sql

```

-- Кредитные программы банка «Решение» (вариант 4): условия приближены к действующим на rbank.by
insert into credit_product (name, kind, currency, rate, min_amount, max_amount, min_term_months, max_term_months, description)
values ('R-деньги mix', 'ANNUITY', 'BYN', 17.65, 500.00, 50000.00, 6, 60,
       'Кредит на потребительские нужды, погашение равными (аннуитетными) платежами'),
       ('R-Онлайн', 'INTEREST_ONLY', 'BYN', 17.65, 300.00, 15000.00, 3, 36,
       'Кредит с ежемесячной уплатой процентов и возвратом всей суммы в конце срока');

insert into mobile_operator (code, name) values
('A1', 'A1'),
('MTS', 'MTC'),
('LIFE', 'life:');

```

Листинг A.5 – credit-service/src/main/java/by/bsuir/bank/credit/CreditServiceApplication.java

```

package by.bsuir.bank.credit;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/** Вместе с пакетом сервиса сканируется общая библиотека: REST-клиенты смежных сервисов и обработчик ошибок. */
@SpringBootApplication(scanBasePackages = {"by.bsuir.bank.credit", "by.bsuir.bank.common"})
public class CreditServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(CreditServiceApplication.class, args);
    }
}

```

Листинг А.6 – credit-service/src/main/java/by/bsuir/bank/credit/domain/Card.java

```
package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import lombok.Getter;
import lombok.Setter;

/** Банковская карта, привязанная к кредитному счёту договора. */
@Entity
@Getter
@Setter
public class Card {

    public static final int MAX_PIN_ATTEMPTS = 3;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @OneToOne(fetch = FetchType.EAGER, optional = false)
    private CreditContract contract;

    private String holder;
    private String pinHash;
    private int failedAttempts;
    private boolean blocked;

    /** Неверный PIN-код: после третьей ошибки подряд карта блокируется. Возвращает число оставшихся попыток. */
    public int registerFailedAttempt() {
        failedAttempts++;
        if (failedAttempts >= MAX_PIN_ATTEMPTS) {
            blocked = true;
        }
        return Math.max(0, MAX_PIN_ATTEMPTS - failedAttempts);
    }

    public void resetAttempts() {
        failedAttempts = 0;
        blocked = false;
    }
}
```

Листинг А.7 – credit-service/src/main/java/by/bsuir/bank/credit/domain/CardNumbers.java

```
package by.bsuir.bank.credit.domain;

/** Номер карты: 16 цифр, префикс банка, порядковый номер и контрольная цифра по алгоритму Луна. */
public final class CardNumbers {

    private static final String BIN = "911238";

    public static String generate(long sequence) {
        String body = BIN + String.format("%09d", sequence);
        return body + luhnDigit(body);
    }

    public static boolean isValid(String number) {
        return number != null && number.matches("\\d{16}")
            && luhnDigit(number.substring(0, 15)) == number.charAt(15) - '0';
    }

    /** Цифры на чётных с конца позициях удваиваются; контрольная цифра дополняет сумму до кратной 10. */
    static int luhnDigit(String body) {
        int sum = 0;
        boolean doubled = true;
        for (int i = body.length() - 1; i >= 0; i--) {
            int digit = body.charAt(i) - '0';
            if (doubled) {
                digit *= 2;
                if (digit > 9) {

```

```

        digit -= 9;
    }
    }
    sum += digit;
    doubled = !doubled;
}
return (10 - sum % 10) % 10;
}

private CardNumbers() {
}
}
}

```

Листинг А.8 – credit-service/src/main/java/by/bsuir/bank/credit/domain/ContractStatus.java

```

package by.bsuir.bank.credit.domain;

public enum ContractStatus {
    ACTIVE, CLOSED
}

```

Листинг А.9 – credit-service/src/main/java/by/bsuir/bank/credit/domain/CreditContract.java

```

package by.bsuir.bank.credit.domain;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Кредитный договор с физическим лицом и его график платежей. */
@Entity
@Getter
@Setter
public class CreditContract {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String number;

    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    private CreditProduct product;

    private Long clientId;
    private String clientName;
    private String currency;
    private BigDecimal amount;
    private BigDecimal rate;
    private int termMonths;
    private LocalDate startDate;
    private LocalDate endDate;
    private String mainAccount;
    private String interestAccount;

    @Enumerated(EnumType.STRING)
    private ContractStatus status;

    private BigDecimal principalPaid = BigDecimal.ZERO;
    private BigDecimal interestPaid = BigDecimal.ZERO;
    private LocalDate closedOn;
}

```

```

@OneToMany(mappedBy = "contract", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderBy("seq")
private List<PaymentItem> schedule = new ArrayList<>();

public boolean isActive() {
    return status == ContractStatus.ACTIVE;
}
}

```

Листинг A.10 – credit-service/src/main/java/by/bsuir/bank/credit/domain/CreditKind.java

```

package by.bsuir.bank.credit.domain;

import lombok.Getter;
import lombok.RequiredArgsConstructor;

/** Вид кредита определяет способ погашения. */
@Getter
@RequiredArgsConstructor
public enum CreditKind {

    /** Ежемесячное погашение долга равными аннуитетными платежами. */
    ANNUITY("Аннуитетные платежи"),

    /** Ежемесячное погашение процентов, вся сумма кредита возвращается в конце срока. */
    INTEREST_ONLY("Проценты ежемесячно, долг в конце срока");

    private final String title;
}

```

Листинг A.11 – credit-service/src/main/java/by/bsuir/bank/credit/domain/CreditProduct.java

```

package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

/** Кредитная программа – запись справочника «вид кредита». */
@Entity
@Getter
@Setter
public class CreditProduct {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Enumerated(EnumType.STRING)
    private CreditKind kind;

    private String currency;
    private BigDecimal rate;
    private BigDecimal minAmount;
    private BigDecimal maxAmount;
    private int minTermMonths;
    private int maxTermMonths;
    private String description;

    public String getKindTitle() {
        return kind.getTitle();
    }
}

```

Листинг А.12 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/MobileOperator.java

```
package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

/** Оператор мобильной связи – получатель платежей из банкомата. */
@Entity
@Getter
@Setter
public class MobileOperator {

    @Id
    private String code;

    private String name;
}
```

Листинг А.13 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/MobilePayment.java

```
package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Платёж за услуги мобильной связи, проведённый через банкомат. */
@Entity
@Getter
@Setter
public class MobilePayment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String cardNumber;
    private String operator;
    private String phone;
    private BigDecimal amount;
    private LocalDate bankDate;
    private String reference;
}
```

Листинг А.14 –

credit-service/src/main/java/by/bsuir/bank/credit/domain/PaymentItem.java

```
package by.bsuir.bank.credit.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToOne;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Строка графика платежей: дата, погашение основного долга, проценты, остаток долга. */
@Entity
@Getter
@Setter
public class PaymentItem {
```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
private CreditContract contract;

private int seq;
private LocalDate dueDate;
private BigDecimal principal;
private BigDecimal interest;
private BigDecimal balanceAfter;
private LocalDate paidOn;

public boolean isPaid() {
    return paidOn != null;
}
}

```

Листинг А.15 – credit-service/src/main/java/by/bsuir/bank/credit/repository/CardRepository.java

```

package by.bsuir.bank.credit.repository;

import by.bsuir.bank.credit.domain.Card;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface CardRepository extends JpaRepository<Card, Long> {

    Optional<Card> findByNumber(String number);

    Optional<Card> findByContractId(Long contractId);
}

```

Листинг А.16 – credit- service/src/main/java/by/bsuir/bank/credit/repository/CreditContractRepository.java

```

package by.bsuir.bank.credit.repository;

import by.bsuir.bank.credit.domain.ContractStatus;
import by.bsuir.bank.credit.domain.CreditContract;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface CreditContractRepository extends JpaRepository<CreditContract, Long> {

    boolean existsByNumber(String number);

    List<CreditContract> findAllByOrderByIdDesc();

    @Query("select c.id from CreditContract c where c.status = :status order by c.id")
    List<Long> findIdsByStatus(ContractStatus status);

    @Query("select max(c.number) from CreditContract c")
    String maxNumber();
}

```

Листинг А.17 – credit- service/src/main/java/by/bsuir/bank/credit/repository/CreditProductRepository.java

```

package by.bsuir.bank.credit.repository;

import by.bsuir.bank.credit.domain.CreditProduct;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CreditProductRepository extends JpaRepository<CreditProduct, Long> {
}

```

Листинг А.18 – credit-service/src/main/java/by/bsuir/bank/credit/repository/MobileOperatorRepository.java

```
package by.bsuir.bank.credit.repository;

import by.bsuir.bank.credit.domain.MobileOperator;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MobileOperatorRepository extends JpaRepository<MobileOperator, String> {
}
```

Листинг А.19 – credit-service/src/main/java/by/bsuir/bank/credit/repository/MobilePaymentRepository.java

```
package by.bsuir.bank.credit.repository;

import by.bsuir.bank.credit.domain.MobilePayment;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MobilePaymentRepository extends JpaRepository<MobilePayment, Long> {
}
```

Листинг А.20 – credit-service/src/main/java/by/bsuir/bank/credit/dto/CashRequest.java

```
package by.bsuir.bank.credit.dto;

import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;

import static by.bsuir.bank.common.api.CommonExceptionHandler.REQUIRED;

/** Выдача наличных с кредитного счёта через кассу. */
public record CashRequest(
    @NotNull(message = REQUIRED)
    @Positive(message = "Сумма должна быть больше нуля")
    @Digits(integer = 12, fraction = 2, message = "Денежная сумма: не более 2 знаков после запятой")
    BigDecimal amount
) {
}
```

Листинг А.21 – credit-service/src/main/java/by/bsuir/bank/credit/dto/CreditDetails.java

```
package by.bsuir.bank.credit.dto;

import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.OperationInfo;

import java.util.List;

/** Договор с графиком платежей, состоянием счетов и проводками из главной книги. */
public record CreditDetails(CreditView contract, ScheduleView schedule, List<AccountInfo> accounts,
    List<OperationInfo> operations) {
}
```

Листинг А.22 – credit-service/src/main/java/by/bsuir/bank/credit/dto/CreditIssued.java

```
package by.bsuir.bank.credit.dto;

/** Результат заключения договора: сам договор и «ПИН-конверт» выпущенной карты (PIN показывается один раз). */
public record CreditIssued(CreditView contract, PinEnvelope card) {
}
```

Листинг А.23 – credit-service/src/main/java/by/bsuir/bank/credit/dto/CreditRequest.java

```

package by.bsuir.bank.credit.dto;

import jakarta.validation.constraints.Digits;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;
import java.time.LocalDate;

import static by.bsuir.bank.common.api.CommonExceptionHandler.REQUIRED;

/** Данные формы заключения кредитного договора. */
public record CreditRequest(

    @NotBlank(message = REQUIRED)
    @Pattern(regexp = "^K-\\d{6}$", message = "Формат номера договора: K-000001")
    String number,

    @NotNull(message = REQUIRED)
    Long productId,

    @NotBlank(message = REQUIRED)
    String currency,

    @NotNull(message = REQUIRED)
    Long clientId,

    @NotNull(message = REQUIRED)
    @Positive(message = "Сумма должна быть больше нуля")
    @Digits(integer = 12, fraction = 2, message = "Денежная сумма: не более 2 знаков после запятой")
    BigDecimal amount,

    @NotNull(message = REQUIRED)
    BigDecimal rate,

    @NotNull(message = REQUIRED)
    @Positive(message = "Срок должен быть больше нуля")
    Integer termMonths,

    @NotNull(message = REQUIRED)
    LocalDate startDate,

    @NotNull(message = REQUIRED)
    LocalDate endDate

) {
}

```

Листинг А.24 – credit-service/src/main/java/by/bsuir/bank/credit/dto/CreditView.java

```

package by.bsuir.bank.credit.dto;

import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CreditContract;

import java.math.BigDecimal;
import java.time.LocalDate;

public record CreditView(
    Long id,
    String number,
    Long productId,
    String productName,
    String kind,
    String kindTitle,
    Long clientId,
    String clientName,
    String currency,
    BigDecimal amount,
    BigDecimal rate,
    int termMonths,
    LocalDate startDate,
    LocalDate endDate,
    String mainAccount,
    String interestAccount,
    String status,
    BigDecimal principalPaid,

```

```

        BigDecimal interestPaid,
        BigDecimal debt,
        LocalDate closedOn,
        String cardNumber,
        boolean cardBlocked
    ) {

        public static CreditView from(CreditContract c, Card card) {
            return new CreditView(c.getId(), c.getNumber(), c.getProduct().getId(), c.getProduct().getName(),
                c.getProduct().getKind().name(), c.getProduct().getKind().getTitle(), c.getClientId(),
                c.getClientName(), c.getCurrency(), c.getAmount(), c.getRate(), c.getTermMonths(),
                c.getStartDate(), c.getEndDate(), c.getMainAccount(), c.getInterestAccount(), c.getStatus().name(),
                c.getPrincipalPaid(), c.getInterestPaid(), c.getAmount().subtract(c.getPrincipalPaid()),
                c.getClosedOn(), card == null ? null : card.getNumber(), card != null && card.isBlocked());
        }
    }
}

```

Листинг A.25 – credit-service/src/main/java/by/bsuir/bank/credit/dto/Meta.java

```

package by.bsuir.bank.credit.dto;

import by.bsuir.bank.common.ledger.CurrencyInfo;

import java.time.LocalDate;
import java.util.List;

/** Данные для формы договора: банковский день, справочник валют, следующий свободный номер договора. */
public record Meta(LocalDate bankDate, List<CurrencyInfo> currencies, String nextNumber) {
}

```

Листинг A.26 – credit-service/src/main/java/by/bsuir/bank/credit/dto/PinEnvelope.java

```

package by.bsuir.bank.credit.dto;

/** «ПИН-конверт»: номер карты и PIN-код в открытом виде. В базе остаётся только хеш PIN-кода. */
public record PinEnvelope(String cardNumber, String pin) {
}

```

Листинг A.27 – credit-service/src/main/java/by/bsuir/bank/credit/dto/ScheduleRequest.java

```

package by.bsuir.bank.credit.dto;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

import java.math.BigDecimal;

import static by.bsuir.bank.common.api.CommonExceptionHandler.REQUIRED;

/** Параметры предварительного расчёта графика платежей. */
public record ScheduleRequest(
    @NotNull(message = REQUIRED) Long productId,
    @NotNull(message = REQUIRED) @Positive BigDecimal amount,
    @NotNull(message = REQUIRED) @Positive @Max(600) Integer termMonths
) {
}

```

Листинг A.28 – credit-service/src/main/java/by/bsuir/bank/credit/dto/ScheduleRow.java

```

package by.bsuir.bank.credit.dto;

import by.bsuir.bank.credit.domain.PaymentItem;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Строка графика платежей. */
public record ScheduleRow(int seq, LocalDate dueDate, BigDecimal principal, BigDecimal interest,
    BigDecimal total, BigDecimal balanceAfter, LocalDate paidOn) {
}

```

```

public ScheduleRow(int seq, LocalDate dueDate, BigDecimal principal, BigDecimal interest, BigDecimal balanceAfter) {
    this(seq, dueDate, principal, interest, principal.add(interest), balanceAfter, null);
}

public static ScheduleRow from(PaymentItem item) {
    return new ScheduleRow(item.getSeq(), item.getDueDate(), item.getPrincipal(), item.getInterest(),
        item.getPrincipal().add(item.getInterest()), item.getBalanceAfter(), item.getPaidOn());
}
}

```

Листинг А.29 – credit-service/src/main/java/by/bsuir/bank/credit/dto/ScheduleView.java

```

package by.bsuir.bank.credit.dto;

import java.math.BigDecimal;
import java.util.List;

/** График платежей с итогами: сумма процентов (переплата) и полная сумма выплат. */
public record ScheduleView(List<ScheduleRow> rows, BigDecimal totalInterest, BigDecimal totalPayment) {

    public static ScheduleView of(List<ScheduleRow> rows) {
        BigDecimal interest = rows.stream().map(ScheduleRow::interest).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal total = rows.stream().map(ScheduleRow::total).reduce(BigDecimal.ZERO, BigDecimal::add);
        return new ScheduleView(rows, interest, total);
    }
}

```

Листинг А.30 – credit-service/src/main/java/by/bsuir/bank/credit/service/CardService.java

```

package by.bsuir.bank.credit.service;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CardNumbers;
import by.bsuir.bank.credit.domain.CreditContract;
import by.bsuir.bank.credit.dto.PinEnvelope;
import by.bsuir.bank.credit.repository.CardRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.security.SecureRandom;

/** Выпуск карт к кредитным счетам и проверка PIN-кодов. */
@Service
@RequiredArgsConstructor
public class CardService {

    private final CardRepository cards;
    private final PasswordEncoder encoder = new BCryptPasswordEncoder();
    private final SecureRandom random = new SecureRandom();

    /** Выпускает карту к договору. PIN-код генерируется случайно и возвращается единственный раз. */
    @Transactional
    public PinEnvelope issue(CreditContract contract) {
        Card card = new Card();
        card.setNumber(CardNumbers.generate(contract.getId()));
        card.setContract(contract);
        card.setHolder(contract.getClientName());
        return assignPin(card);
    }

    /** Перевыпуск PIN-кода: заодно снимает блокировку карты после трёх неверных попыток. */
    @Transactional
    public PinEnvelope reissuePin(Long contractId) {
        Card card = cards.findByContractId(contractId)
            .orElseThrow(() -> BankException.notFound("К договору не выпущена карта"));
        card.resetAttempts();
        return assignPin(card);
    }

    public boolean matches(Card card, String pin) {
        return pin != null && encoder.matches(pin, card.getPinHash());
    }
}

```

```

private PinEnvelope assignPin(Card card) {
    String pin = String.format("%04d", random.nextInt(10_000));
    card.setPinHash(encoder.encode(pin));
    cards.save(card);
    return new PinEnvelope(card.getNumber(), pin);
}
}

```

Листинг А.31 –

credit-service/src/main/java/by/bsuir/bank/credit/service/CreditPostings.java

```

package by.bsuir.bank.credit.service;

import by.bsuir.bank.common.ledger.Entry;
import by.bsuir.bank.common.ledger.Posting;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

/**
 * Проводки процесса «Кредитная программа» – в точности по упрощённой схеме из задания.
 * Касса, текущий (кредитный) и процентный счета клиента – активные; фонд развития – пассивный.
 */
public final class CreditPostings {

    /** Выдача кредита: банк выделяет сумму из фонда развития на текущий счёт клиента. */
    public static Posting issue(String fund, String main, BigDecimal amount) {
        return Posting.of("Выделение кредита банком", Entry.debit(fund, amount), Entry.debit(main, amount));
    }

    /** Снятие наличных: с текущего счёта в кассу и из кассы клиенту (в отделении или в банкомате). */
    public static List<Posting> cashOut(String cash, String main, BigDecimal amount) {
        return List.of(
            Posting.of("Перевод кредита в кассу", Entry.credit(main, amount), Entry.debit(cash, amount)),
            Posting.of("Получение кредита через кассу", Entry.credit(cash, amount));
        );
    }

    /** Очередной платёж по графику: уплата процентов и погашение части основного долга. */
    public static List<Posting> payment(String cash, String fund, String main, String interestAccount,
        BigDecimal interest, BigDecimal principal, boolean last) {
        List<Posting> postings = new ArrayList<>();
        if (interest.signum() > 0) {
            postings.add(Posting.of("Внесение процентов в кассу", Entry.debit(cash, interest)));
            postings.add(Posting.of("Перевод процентов из кассы", Entry.credit(cash, interest), Entry.debit(interestAccount,
                interest)));
            postings.add(Posting.of("Начисление процентов банком", Entry.credit(interestAccount, interest), Entry.credit(fund,
                interest)));
        }
        if (principal.signum() > 0) {
            postings.add(Posting.of("Внесение денег в кассу", Entry.debit(cash, principal)));
            postings.add(Posting.of("Погашение долга за кредит из кассы", Entry.credit(cash, principal), Entry.debit(main,
                principal)));
            postings.add(Posting.of(last ? "Окончание кредита" : "Возврат части кредита в фонд банка",
                Entry.credit(main, principal), Entry.credit(fund, principal)));
        }
        return postings;
    }

    /** Оплата услуг связи с кредитного счёта на расчётный счёт оператора. */
    public static Posting mobilePayment(String main, String operatorAccount, String description, BigDecimal amount) {
        return Posting.of(description, Entry.credit(main, amount), Entry.credit(operatorAccount, amount));
    }

    private CreditPostings() {
    }
}

```

Листинг А.32 –

credit-service/src/main/java/by/bsuir/bank/credit/service/CreditService.java

```

package by.bsuir.bank.credit.service;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;

```

```

import by.bsuir.bank.common.ledger.Batch;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OpenAccount;
import by.bsuir.bank.common.ledger.Posting;
import by.bsuir.bank.credit.domain.ContractStatus;
import by.bsuir.bank.credit.domain.CreditContract;
import by.bsuir.bank.credit.domain.CreditProduct;
import by.bsuir.bank.credit.domain.PaymentItem;
import by.bsuir.bank.credit.dto.CreditDetails;
import by.bsuir.bank.credit.dto.CreditIssued;
import by.bsuir.bank.credit.dto.CreditRequest;
import by.bsuir.bank.credit.dto.CreditView;
import by.bsuir.bank.credit.dto.Meta;
import by.bsuir.bank.credit.dto.PinEnvelope;
import by.bsuir.bank.credit.dto.ScheduleRequest;
import by.bsuir.bank.credit.dto.ScheduleRow;
import by.bsuir.bank.credit.dto.ScheduleView;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import by.bsuir.bank.credit.repository.CreditProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.HttpClientErrorException;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Service
@RequiredArgsConstructor
public class CreditService {

    public static final String MAIN_CHART_CODE = "2400";
    public static final String INTEREST_CHART_CODE = "2470";

    private static final DateTimeFormatter RU_DATE = DateTimeFormatter.ofPattern("dd.MM.yyyy");

    private final CreditContractRepository contracts;
    private final CreditProductRepository products;
    private final CardRepository cards;
    private final CardService cardService;
    private final ClientApi clients;
    private final LedgerApi ledger;

    public Meta meta() {
        String last = contracts.maxNumber();
        int next = last == null ? 1 : Integer.parseInt(last.substring(2)) + 1;
        return new Meta(ledger.bankDay().date(), ledger.currencies(), String.format("K-%06d", next));
    }

    /** Предварительный расчёт графика платежей для формы договора. */
    @Transactional(readOnly = true)
    public ScheduleView preview(ScheduleRequest request) {
        CreditProduct product = product(request.productId());
        return ScheduleView.of(ScheduleCalculator.build(product.getKind(), request.amount(), product.getRate(),
            request.termMonths(), ledger.bankDay().date()));
    }

    @Transactional(readOnly = true)
    public List<CreditView> list() {
        return contracts.findAllByIdOrderByDesc().stream().map(this::view).toList();
    }

    @Transactional(readOnly = true)
    public CreditDetails details(Long id) {
        CreditContract contract = find(id);
        List<ScheduleRow> rows = contract.getSchedule().stream().map(ScheduleRow::from).toList();
        return new CreditDetails(view(contract), ScheduleView.of(rows),
            ledger.accounts(contract.getNumber()), ledger.operations(contract.getNumber()));
    }

    /**
     * Заключение договора: проверка условий программы, открытие текущего и процентного счетов,
     * выделение кредита из фонда развития, расчёт графика платежей и выпуск карты к счёту.
     */

```

```

@Transactional
public CreditIssued open(CreditRequest request) {
    CreditProduct product = product(request.productId());
    LocalDate today = ledger.bankDay().date();
    checkTerms(request, product, today);
    if (contracts.existsByNumber(request.number())) {
        throw new BankException(HttpStatus.CONFLICT, "DUPLICATE_CONTRACT", "Договор уже существует",
            Map.of("number", "Договор с таким номером уже заключён"));
    }
    ClientInfo client = findClient(request.clientId());

    String number = request.number();
    String currency = product.getCurrency();
    String main = ledger.open(new OpenAccount(MAIN_CHART_CODE, currency,
        client.id().intValue(), client.fullName(), number)).number();
    String interest = ledger.open(new OpenAccount(INTEREST_CHART_CODE, currency,
        client.id().intValue(), client.fullName(), number)).number();
    post(key(number, "ISSUE"), number, today, List.of(CreditPostings.issue(fund(currency), main, request.amount())));

    CreditContract contract = new CreditContract();
    contract.setNumber(number);
    contract.setProduct(product);
    contract.setClientId(client.id());
    contract.setClientName(client.fullName());
    contract.setCurrency(currency);
    contract.setAmount(request.amount());
    contract.setRate(product.getRate());
    contract.setTermMonths(request.termMonths());
    contract.setStartDate(today);
    contract.setEndDate(request.endDate());
    contract.setMainAccount(main);
    contract.setInterestAccount(interest);
    contract.setStatus(ContractStatus.ACTIVE);
    for (ScheduleRow row : ScheduleCalculator.build(product.getKind(), request.amount(), product.getRate(),
        request.termMonths(), today)) {
        PaymentItem item = new PaymentItem();
        item.setContract(contract);
        item.setSeq(row.seq());
        item.setDueDate(row.dueDate());
        item.setPrincipal(row.principal());
        item.setInterest(row.interest());
        item.setBalanceAfter(row.balanceAfter());
        contract.getSchedule().add(item);
    }
    contracts.saveAndFlush(contract);
    PinEnvelope card = cardService.issue(contract);
    return new CreditIssued(view(contract), card);
}

/** Контроль корректности условий договора относительно выбранной кредитной программы. */
private void checkTerms(CreditRequest request, CreditProduct product, LocalDate today) {
    Map<String, String> errors = new LinkedHashMap<>();
    if (!product.getCurrency().equals(request.currency())) {
        errors.put("productId", "Программа недоступна в валюте " + request.currency());
    }
    if (request.amount().compareTo(product.getMinAmount()) < 0 || request.amount().compareTo(product.getMaxAmount()) > 0) {
        errors.put("amount", "Сумма кредита по программе: от " + product.getMinAmount()
            + " до " + product.getMaxAmount() + " " + product.getCurrency());
    }
    if (request.rate().compareTo(product.getRate()) != 0) {
        errors.put("rate", "Ставка по программе — " + product.getRate() + " %");
    }
    if (request.termMonths() < product.getMinTermMonths() || request.termMonths() > product.getMaxTermMonths()) {
        errors.put("termMonths", "Срок по программе: от " + product.getMinTermMonths()
            + " до " + product.getMaxTermMonths() + " мес.");
    }
    if (!request.startDate().equals(today)) {
        errors.put("startDate", "Договор заключается текущим банковским днём: " + RU_DATE.format(today));
    }
    if (!request.endDate().equals(request.startDate().plusMonths(request.termMonths()))) {
        errors.put("endDate", "Дата окончания не соответствует сроку договора");
    }
    if (!errors.isEmpty()) {
        throw BankException.fields(errors);
    }
}

/**
 * Снятие наличных с кредитного счёта — в кассе отделения или в банкомате.
 * Возвращает состояние счёта после операции. Метод намеренно не транзакционный: локальных данных
 * он не меняет, а отказ (нехватка средств) не должен помечать на откат транзакцию вызывающего кода.
 */

```

```

public AccountInfo cashOut(Long id, BigDecimal amount) {
    CreditContract contract = find(id);
    BigDecimal available = ledger.account(contract.getMainAccount()).balance();
    if (available.compareTo(amount) < 0) {
        throw BankException.conflict("INSUFFICIENT_FUNDS",
            "Недостаточно средств на счёте: доступно " + available + " " + contract.getCurrency());
    }
    post(key(contract.getNumber(), "CASH-" + UUID.randomUUID()), contract.getNumber(), null,
        CreditPostings.cashOut(cash(contract.getCurrency()), contract.getMainAccount(), amount));
    return ledger.account(contract.getMainAccount());
}

/**
 * Обработка договора при открытии банковского дня date: проводятся все платежи графика,
 * срок которых наступил. Считается, что клиент платит без задержки. Ключ пакета проводок
 * привязан к номеру платежа, поэтому повторная обработка дня не проводит платёж дважды.
 */
@Transactional
public List<String> processDay(Long id, LocalDate date) {
    CreditContract contract = find(id);
    List<String> events = new ArrayList<>();
    if (!contract.isActive()) {
        return events;
    }
    String number = contract.getNumber();
    String currency = contract.getCurrency();
    for (PaymentItem item : contract.getSchedule()) {
        if (item.isPaid() || item.getDueDate().isAfter(date)) {
            continue;
        }
        boolean last = item.getSeq() == contract.getTermMonths();
        post(key(number, "PAY-" + item.getSeq()), number, date, CreditPostings.payment(cash(currency), fund(currency),
            contract.getMainAccount(), contract.getInterestAccount(), item.getInterest(), item.getPrincipal(), last));
        item.setPaidOn(date);
        contract.setInterestPaid(contract.getInterestPaid().add(item.getInterest()));
        contract.setPrincipalPaid(contract.getPrincipalPaid().add(item.getPrincipal()));
        events.add(RU_DATE.format(date) + ": " + number + " – платёж №" + item.getSeq() + ": проценты " + item.getInterest()
            + ", основной долг " + item.getPrincipal() + " " + currency);
    }
    if (contract.getSchedule().stream().allMatch(PaymentItem::isPaid)) {
        contract.setStatus(ContractStatus.CLOSED);
        contract.setClosedOn(date);
        events.add(RU_DATE.format(date) + ": " + number + " – кредит погашен, договор закрыт");
    }
    return events;
}

private ClientInfo findClient(Long clientId) {
    try {
        return clients.get(clientId);
    } catch (HttpClientErrorException.NotFound e) {
        throw BankException.fields(Map.of("clientId", "Клиент не найден в модуле «Клиенты»"));
    }
}

private CreditProduct product(Long id) {
    return products.findById(id)
        .orElseThrow(() -> BankException.fields(Map.of("productId", "Вид кредита отсутствует в справочнике")));
}

private CreditView view(CreditContract contract) {
    return CreditView.from(contract, cards.findByContractId(contract.getId()).orElse(null));
}

private void post(String batchKey, String contractRef, LocalDate date, List<Posting> postings) {
    ledger.post(new Batch(batchKey, contractRef, date, postings));
}

private static String key(String number, String step) {
    return "CRD-" + number + "-" + step;
}

private String cash(String currency) {
    return ledger.systemAccount(LedgerApi.CASH, currency).number();
}

private String fund(String currency) {
    return ledger.systemAccount(LedgerApi.FUND, currency).number();
}

private CreditContract find(Long id) {
    return contracts.findById(id).orElseThrow(() -> BankException.notFound("Кредитный договор не найден"));
}

```

```

    }
}

```

Листинг А.33 –

credit-service/src/main/java/by/bsuir/bank/credit/service/DayCloseProcessor.java

```

package by.bsuir.bank.credit.service;

import by.bsuir.bank.credit.domain.ContractStatus;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Участие сервиса кредитов в процедуре «Закрытие банковского дня»: каждый договор – в своей транзакции. */
@Component
@RequiredArgsConstructor
public class DayCloseProcessor {

    private final CreditContractRepository contracts;
    private final CreditService credits;

    public List<String> process(LocalDate date) {
        List<String> events = new ArrayList<>();
        for (Long id : contracts.findIdsByStatus(ContractStatus.ACTIVE)) {
            events.addAll(credits.processDay(id, date));
        }
        return events;
    }
}

```

Листинг А.34 –

credit-service/src/main/java/by/bsuir/bank/credit/service/ScheduleCalculator.java

```

package by.bsuir.bank.credit.service;

import by.bsuir.bank.credit.domain.CreditKind;
import by.bsuir.bank.credit.dto.ScheduleRow;

import java.math.BigDecimal;
import java.math.MathContext;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Расчёт графика платежей по кредиту. Месячная ставка i = годовая / 12 / 100. */
public final class ScheduleCalculator {

    private static final MathContext PRECISION = MathContext.DECIMAL64;
    private static final BigDecimal MONTHS_PERCENT = BigDecimal.valueOf(1200);

    public static List<ScheduleRow> build(CreditKind kind, BigDecimal amount, BigDecimal rate, int months, LocalDate start) {
        return kind == CreditKind.ANNUITY ? annuity(amount, rate, months, start) : interestOnly(amount, rate, months, start);
    }

    /** Аннуитетный платёж:  $A = S * i / (1 - (1 + i)^{-n})$ . */
    public static BigDecimal annuityPayment(BigDecimal amount, BigDecimal rate, int months) {
        BigDecimal i = rate.divide(MONTHS_PERCENT, PRECISION);
        if (i.signum() == 0) {
            return amount.divide(BigDecimal.valueOf(months), 2, RoundingMode.HALF_UP);
        }
        BigDecimal growth = BigDecimal.ONE.add(i).pow(months, PRECISION);
        return amount.multiply(i).multiply(growth)
            .divide(growth.subtract(BigDecimal.ONE), 2, RoundingMode.HALF_UP);
    }

    /**
     * Аннуитет: платёж постоянен, проценты начисляются на остаток долга, остальное гасит долг.
     * Последний платёж закрывает остаток целиком, вбирая накопленную погрешность округления.
     */
    private static List<ScheduleRow> annuity(BigDecimal amount, BigDecimal rate, int months, LocalDate start) {
        BigDecimal payment = annuityPayment(amount, rate, months);
        BigDecimal balance = amount;
        List<ScheduleRow> rows = new ArrayList<>();
    }
}

```

```

        for (int k = 1; k <= months; k++) {
            BigDecimal interest = monthlyInterest(balance, rate);
            BigDecimal principal = k == months ? balance : payment.subtract(interest).min(balance);
            balance = balance.subtract(principal);
            rows.add(new ScheduleRow(k, start.plusMonths(k), principal, interest, balance));
        }
        return rows;
    }

    /** Ежемесячно уплачиваются только проценты на всю сумму, основной долг возвращается последним платежом. */
    private static List<ScheduleRow> interestOnly(BigDecimal amount, BigDecimal rate, int months, LocalDate start) {
        BigDecimal interest = monthlyInterest(amount, rate);
        List<ScheduleRow> rows = new ArrayList<>();
        for (int k = 1; k <= months; k++) {
            boolean last = k == months;
            rows.add(new ScheduleRow(k, start.plusMonths(k), last ? amount : BigDecimal.ZERO.setScale(2), interest,
                last ? BigDecimal.ZERO.setScale(2) : amount));
        }
        return rows;
    }

    private static BigDecimal monthlyInterest(BigDecimal balance, BigDecimal rate) {
        return balance.multiply(rate).divide(MONTHS_PERCENT, 2, RoundingMode.HALF_UP);
    }

    private ScheduleCalculator() {
    }
}

```

Листинг А.35 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmConfig.java

```

package by.bsuir.bank.credit.atm;

import by.bsuir.bank.common.HttpApis;
import by.bsuir.bank.common.ServiceUrls;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;

@Configuration
public class AtmConfig {

    @Bean
    public DepositApi depositApi(RestClient.Builder builder, ServiceUrls urls) {
        return HttpApis.proxy(builder, urls.deposits(), DepositApi.class);
    }
}

```

Листинг А.36 – credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmProcessor.java

```

package by.bsuir.bank.credit.atm;

import by.bsuir.bank.common.api.ApiError;
import by.bsuir.bank.common.api.ApiException;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.Batch;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OpenAccount;
import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CreditContract;
import by.bsuir.bank.credit.domain.MobileOperator;
import by.bsuir.bank.credit.domain.MobilePayment;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.MobileOperatorRepository;
import by.bsuir.bank.credit.repository.MobilePaymentRepository;
import by.bsuir.bank.credit.service.CardService;
import by.bsuir.bank.credit.service.CreditPostings;
import by.bsuir.bank.credit.service.CreditService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestClientResponseException;

import java.math.BigDecimal;
import java.time.LocalDate;

```

```

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

/**
 * Банковская сторона протокола «банк – банкомат». Каждая транзакция банкомата содержит номер карты,
 * PIN-код и описание операции; банк заново проверяет карту и PIN-код при каждом запросе.
 */
@Service
@RequiredArgsConstructor
public class AtmProcessor {

    public static final String OPERATOR_CHART_CODE = "3012";

    private static final String PHONE_FORMAT = "^0(25|29|33|44)\\d{7}$";

    private final CardRepository cards;
    private final CardService cardService;
    private final CreditService credits;
    private final MobileOperatorRepository operators;
    private final MobilePaymentRepository payments;
    private final LedgerApi ledger;
    private final DepositApi deposits;

    @Transactional
    public AtmResponse handle(AtmRequest request) {
        Map<String, String> tx = new LinkedHashMap<>();
        request.items().forEach(item -> tx.put(item.name(), item.value()));
        String operation = tx.get("OPERATION");
        if (tx.get("CARD") == null || tx.get("PIN") == null || operation == null) {
            return AtmResponse.error("BAD_REQUEST", "Неполное описание транзакции");
        }

        Card card = cards.findByNumber(tx.get("CARD")).orElse(null);
        if (card == null) {
            return AtmResponse.error("CARD_NOT_FOUND", "Карта не обслуживается банком");
        }
        if (card.isBlocked()) {
            return AtmResponse.error("CARD_BLOCKED", "Карта заблокирована. Обратитесь в банк");
        }
        if (!cardService.matches(card, tx.get("PIN"))) {
            int left = card.registerFailedAttempt();
            return left == 0
                ? AtmResponse.error("CARD_BLOCKED", "PIN-код трижды введён неверно. Карта заблокирована")
                : AtmResponse.error("WRONG_PIN", "Неверный PIN-код. Осталось попыток: " + left, Map.of("attemptsLeft", left));
        }
        card.setFailedAttempts(0);

        try {
            return switch (operation) {
                case "AUTHORIZE" -> AtmResponse.ok("Авторизация выполнена", Map.of("holder", card.getHolder()));
                case "BALANCE" -> balance(card);
                case "DEPOSIT_BALANCE" -> depositBalance(card);
                case "WITHDRAW" -> withdraw(card, tx.get("AMOUNT"));
                case "PAYMENT" -> payment(card, tx.get("OPERATOR"), tx.get("PHONE"), tx.get("AMOUNT"));
                default -> AtmResponse.error("UNKNOWN_OPERATION", "Операция не поддерживается");
            };
        } catch (BankException e) {
            return AtmResponse.error(e.getCode(), e.getMessage());
        } catch (RestClientResponseException e) {
            return rejected(e);
        } catch (RestClientException e) {
            return AtmResponse.error("SERVICE_UNAVAILABLE", "Операция временно недоступна");
        }
    }

    /** Отказ смежного сервиса (например, главной книги) передаётся банкомату с исходным кодом и текстом. */
    private static AtmResponse rejected(RestClientResponseException e) {
        try {
            ApiError error = e.getResponseBodyAs(ApiError.class);
            if (error != null && error.code() != null && error.message() != null) {
                return AtmResponse.error(error.code(), error.message());
            }
        } catch (RuntimeException ignored) {
            // тело ответа не в формате ApiError
        }
        return AtmResponse.error("BANK_ERROR", "Операция отклонена банком");
    }

    public List<MobileOperator> operators() {
        return operators.findAll();
    }

```

```

}

private AtmResponse balance(Card card) {
    CreditContract contract = card.getContract();
    AccountInfo account = ledger.account(contract.getMainAccount());
    return AtmResponse.ok("Остаток кредитного счёта", receipt(card, Map.of(
        "account", account.number(), "balance", account.balance(), "currency", account.currency())));
}

private AtmResponse depositBalance(Card card) {
    List<Map<String, Object>> active = deposits.byClient(card.getContract().getClientId()).stream()
        .filter(deposit -> "ACTIVE".equals(deposit.status()))
        .map(deposit -> Map.<String, Object>.of("number", deposit.number(), "product", deposit.productName(),
            "amount", deposit.amount(), "currency", deposit.currency()))
        .toList();
    return AtmResponse.ok(active.isEmpty() ? "Действующих вкладов нет" : "Остатки депозитных счетов",
        receipt(card, Map.of("deposits", active)));
}

private AtmResponse withdraw(Card card, String text) {
    if (text == null || !text.matches("\\d{1,9}") || Long.parseLong(text) == 0) {
        return AtmResponse.error("INVALID_AMOUNT", "Сумма должна быть целым положительным числом");
    }
    BigDecimal amount = new BigDecimal(text);
    AccountInfo account = credits.cashOut(card.getContract().getId(), amount);
    return AtmResponse.ok("Заберите деньги", receipt(card, Map.of(
        "amount", amount, "balance", account.balance(), "currency", account.currency())));
}

private AtmResponse payment(Card card, String operatorCode, String phone, String text) {
    MobileOperator operator = operatorCode == null ? null : operators.findById(operatorCode).orElse(null);
    if (operator == null) {
        return AtmResponse.error("UNKNOWN_OPERATOR", "Оператор связи не найден");
    }
    if (phone == null || !phone.matches(PHONE_FORMAT)) {
        return AtmResponse.error("INVALID_PHONE", "Номер телефона: 10 цифр, например 0291234567");
    }
    if (text == null || !text.matches("\\d{1,9}(\\.\\d{1,2})?") || new BigDecimal(text).signum() == 0) {
        return AtmResponse.error("INVALID_AMOUNT", "Сумма платежа должна быть положительным числом");
    }
    BigDecimal amount = new BigDecimal(text);
    CreditContract contract = card.getContract();
    String currency = contract.getCurrency();
    AccountInfo main = ledger.account(contract.getMainAccount());
    if (main.balance().compareTo(amount) < 0) {
        return AtmResponse.error("INSUFFICIENT_FUNDS",
            "Недостаточно средств на счёте: доступно " + main.balance() + " " + currency);
    }

    // расчётный счёт оператора открывается при первом платеже; повторный запрос вернёт тот же счёт
    String operatorAccount = ledger.open(new OpenAccount(OPERATOR_CHART_CODE, currency, null,
        "Расчётный счёт оператора " + operator.getName(), "OPERATOR-" + operator.getCode() + "-" + currency)).number();
    String reference = UUID.randomUUID().toString().substring(0, 8).toUpperCase();
    LocalDate date = ledger.bankDay().date();
    ledger.post(Batch.of("ATM-PAYMENT-" + reference, contract.getNumber(), date,
        CreditPostings.mobilePayment(main.number(), operatorAccount,
            "Оплата услуг связи " + operator.getName() + ", номер " + phone, amount)));

    MobilePayment payment = new MobilePayment();
    payment.setCardNumber(card.getNumber());
    payment.setOperator(operator.getCode());
    payment.setPhone(phone);
    payment.setAmount(amount);
    payment.setBankDate(date);
    payment.setReference(reference);
    payments.save(payment);

    return AtmResponse.ok("Платёж принят", receipt(card, Map.of(
        "operator", operator.getName(), "phone", phone, "amount", amount, "currency", currency,
        "balance", main.balance().subtract(amount), "reference", reference)));
}

/** Общие реквизиты для чека: банковский день и держатель карты. */
private Map<String, Object> receipt(Card card, Map<String, Object> details) {
    Map<String, Object> data = new LinkedHashMap<>(details);
    data.put("bankDate", ledger.bankDay().date().toString());
    data.put("holder", card.getHolder());
    return data;
}
}

```

Листинг А.37 –

credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmRequest.java

```
package by.bsuir.bank.credit.atm;

import jakarta.validation.constraints.NotEmpty;

import java.util.List;

/**
 * Запрос банкомата – содержимое его «внутреннего списка»: упорядоченные пары «имя – значение».
 * Имена элементов: CARD, PIN, OPERATION, AMOUNT, OPERATOR, PHONE.
 */
public record AtmRequest(@NotEmpty List<Item> items) {

    public record Item(String name, String value) {
    }
}
```

Листинг А.38 –

credit-service/src/main/java/by/bsuir/bank/credit/atm/AtmResponse.java

```
package by.bsuir.bank.credit.atm;

import java.util.Map;

/** Ответ банка банкомату: статус OK/ERROR, код и текст для экрана, данные для чека. */
public record AtmResponse(String status, String code, String message, Map<String, Object> data) {

    public static AtmResponse ok(String message, Map<String, Object> data) {
        return new AtmResponse("OK", "OK", message, data);
    }

    public static AtmResponse error(String code, String message) {
        return new AtmResponse("ERROR", code, message, Map.of());
    }

    public static AtmResponse error(String code, String message, Map<String, Object> data) {
        return new AtmResponse("ERROR", code, message, data);
    }
}
```

Листинг А.39 –

credit-service/src/main/java/by/bsuir/bank/credit/atm/DepositApi.java

```
package by.bsuir.bank.credit.atm;

import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.service.annotation.GetExchange;
import org.springframework.web.service.annotation.HttpExchange;

import java.math.BigDecimal;
import java.util.List;

/** REST-клиент сервиса депозитов: вклады клиента для операции банкомата «Остаток депозитного счёта». */
@HttpExchange("/api/deposits")
public interface DepositApi {

    record DepositInfo(String number, String productName, String currency, BigDecimal amount, String status) {
    }

    @GetExchange
    List<DepositInfo> byClient(@RequestParam Long clientId);
}
```

Листинг А.40 –

credit-service/src/main/java/by/bsuir/bank/credit/web/AtmController.java

```
package by.bsuir.bank.credit.web;

import by.bsuir.bank.credit.atm.AtmProcessor;
import by.bsuir.bank.credit.atm.AtmRequest;
import by.bsuir.bank.credit.atm.AtmResponse;
import by.bsuir.bank.credit.domain.MobileOperator;
import jakarta.validation.Valid;
```

```

import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Точка входа протокола «банк — банкомат». */
@RestController
@RequestMapping("/api/atm")
@RequiredArgsConstructor
public class AtmController {

    private final AtmProcessor processor;

    @PostMapping("/transactions")
    public AtmResponse transaction(@Valid @RequestBody AtmRequest request) {
        return processor.handle(request);
    }

    @GetMapping("/operators")
    public List<MobileOperator> operators() {
        return processor.operators();
    }
}

```

Листинг А.41 — credit-service/src/main/java/by/bsuir/bank/credit/web/CreditController.java

```

package by.bsuir.bank.credit.web;

import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.credit.dto.CashRequest;
import by.bsuir.bank.credit.dto.CreditDetails;
import by.bsuir.bank.credit.dto.CreditIssued;
import by.bsuir.bank.credit.dto.CreditRequest;
import by.bsuir.bank.credit.dto.CreditView;
import by.bsuir.bank.credit.dto.PinEnvelope;
import by.bsuir.bank.credit.dto.ScheduleRequest;
import by.bsuir.bank.credit.dto.ScheduleView;
import by.bsuir.bank.credit.service.CardService;
import by.bsuir.bank.credit.service.CreditService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/api/credits")
@RequiredArgsConstructor
public class CreditController {

    private final CreditService service;
    private final CardService cards;

    @GetMapping
    public List<CreditView> list() {
        return service.list();
    }

    @GetMapping("/{id}")
    public CreditDetails details(@PathVariable Long id) {
        return service.details(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public CreditIssued open(@Valid @RequestBody CreditRequest request) {
        return service.open(request);
    }
}

```

```

    /** График платежей до заключения договора. */
    @PostMapping("/schedule")
    public ScheduleView schedule(@Valid @RequestBody ScheduleRequest request) {
        return service.preview(request);
    }

    /** Выдача наличных с кредитного счёта через кассу банка. */
    @PostMapping("/{id}/cash")
    public AccountInfo cash(@PathVariable Long id, @Valid @RequestBody CashRequest request) {
        return service.cashOut(id, request.amount());
    }

    /** Перевыпуск PIN-кода карты (и разблокировка после трёх неверных попыток). */
    @PostMapping("/{id}/pin")
    public PinEnvelope reissuePin(@PathVariable Long id) {
        return cards.reissuePin(id);
    }
}

```

Листинг А.42 – credit-service/src/main/java/by/bsuir/bank/credit/web/DayCloseController.java

```

package by.bsuir.bank.credit.web;

import by.bsuir.bank.credit.service.DayCloseProcessor;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDate;
import java.util.List;

/** Внутренний REST для сервиса «Счета»: обработка открытия нового банковского дня. */
@RestController
@RequiredArgsConstructor
public class DayCloseController {

    private final DayCloseProcessor processor;

    public record DayOpened(LocalDate date) {
    }

    @PostMapping("/api/internal/day-close")
    public List<String> dayOpened(@RequestBody DayOpened event) {
        return processor.process(event.date());
    }
}

```

Листинг А.43 – credit-service/src/main/java/by/bsuir/bank/credit/web/ReferenceController.java

```

package by.bsuir.bank.credit.web;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.BankDayInfo;
import by.bsuir.bank.common.ledger.DayCloseInfo;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.credit.domain.CreditProduct;
import by.bsuir.bank.credit.dto.Meta;
import by.bsuir.bank.credit.repository.CreditProductRepository;
import by.bsuir.bank.credit.service.CreditService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/** Справочные данные формы договора. Клиенты и банковский день запрашиваются у смежных сервисов. */
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor

```

```

public class ReferenceController {

    private final CreditProductRepository products;
    private final CreditService credits;
    private final ClientApi clients;
    private final LedgerApi ledger;

    @GetMapping("/products")
    public List<CreditProduct> products() {
        return products.findAll(Sort.by("id"));
    }

    @GetMapping("/clients")
    public List<ClientInfo> clients() {
        return clients.list();
    }

    @GetMapping("/meta")
    public Meta meta() {
        return credits.meta();
    }

    @GetMapping("/bank-day")
    public BankDayInfo bankDay() {
        return ledger.bankDay();
    }

    @PostMapping("/bank-day/close")
    public DayCloseInfo closeDay(@RequestParam(defaultValue = "1") int days) {
        return ledger.closeDay(days);
    }
}

```

Листинг А.44 – credit-service/src/main/resources/static/index.html

```

<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Кредиты – Банк «Решение»</title>
  <link rel="stylesheet" href="/ui/bank.css">
</head>
<body>
  <div id="app" v-cloak></div>

  <script src="/webjars/vue/dist/vue.global.prod.js"></script>
  <script src="/ui/bank.js"></script>
  <script src="/app.js"></script>
</body>
</html>

```

Листинг А.45 – credit-service/src/main/resources/static/app.js

```

/* Web-клиент сервиса кредитов: список договоров, форма договора с графиком платежей, карточка договора. */

const REQUIRED = 'Обязательное поле';

/* Дата окончания: как LocalDate.plusMonths – день месяца сохраняется либо прижимается к концу месяца. */
function addMonths(iso, months) {
    const [year, month, day] = iso.split('-').map(Number);
    const first = new Date(year, month - 1 + months, 1);
    const lastDay = new Date(first.getFullYear(), first.getMonth() + 1, 0).getDate();
    const pad = n => String(n).padStart(2, '0');
    return first.getFullYear() + '-' + pad(first.getMonth() + 1) + '-' + pad(Math.min(day, lastDay));
}

const scheduleTable = {
    props: ['schedule', 'currency'],
    methods: { money: Bank.money, date: Bank.date },
    template: `
      <table id="schedule">
        <thead>
          <tr><th>№</th><th>Дата платежа</th><th class="num">Основной долг</th><th class="num">Проценты</th>
            <th class="num">Платёж</th><th class="num">Остаток долга</th><th>Оплачен</th></tr>
        </thead>
        <tbody>
          <tr v-for="row in schedule.rows" :key="row.seq">
            <td>{{ row.seq }}</td>

```

```

        <td>{{ date(row.dueDate) }}</td>
        <td class="num">{{ money(row.principal) }}</td>
        <td class="num">{{ money(row.interest) }}</td>
        <td class="num"><b>{{ money(row.total) }}</b></td>
        <td class="num">{{ money(row.balanceAfter) }}</td>
        <td><span v-if="row.paidOn" class="badge ok">{{ date(row.paidOn) }}</span></td>
    </tr>
</tbody>
<tfoot>
    <tr><td colspan="3">Итого, {{ currency }}</td><td class="num">{{ money(schedule.totalInterest) }}</td>
        <td class="num">{{ money(schedule.totalPayment) }}</td><td colspan="2"></td></tr>
</tfoot>
</table>',
};

const app = Vue.createApp({
  data() {
    return {
      view: 'list', // list | form | details
      credits: [],
      products: [],
      clients: [],
      meta: { bankDate: null, currencies: [], nextNumber: '' },
      form: {},
      errors: {},
      formError: '',
      preview: null,
      details: null,
      envelope: null, // «ПИН-конверт» — показывается один раз после выпуска карты
      cashAmount: '',
      notice: '',
      error: '',
      events: [],
      saving: false,
    };
  },
  computed: {
    availableProducts() {
      return this.products.filter(p => p.currency === this.form.currency);
    },
    product() {
      return this.products.find(p => p.id === this.form.productId) || null;
    },
    endDate() {
      const months = Number(this.form.termMonths);
      return this.meta.bankDate && months > 0 && months <= 600 ? addMonths(this.meta.bankDate, months) : null;
    },
    mainAccount() {
      return this.details && this.details.accounts.find(a => a.number === this.details.contract.mainAccount);
    },
  },
  async mounted() {
    try {
      this.products = await Bank.api('GET', '/api/products');
      await this.reload();
    } catch (e) {
      this.error = e.message;
    }
  },
  methods: {
    async reload() {
      this.credits = await Bank.api('GET', '/api/credits');
    },
    async openForm() {
      this.error = '';
      try {
        this.meta = await Bank.api('GET', '/api/meta');
        this.clients = await Bank.api('GET', '/api/clients');
      } catch (e) {
        this.error = e.message;
        return;
      }
      this.form = { clientId: null, currency: 'BYN', productId: null, number: this.meta.nextNumber, amount: '', termMonths: '' };
      this.errors = {};
      this.formError = '';
      this.notice = '';
      this.envelope = null;
      this.preview = null;
      this.view = 'form';
    },
    amount() {
      return Number(String(this.form.amount).trim().replace(',', '.'));
    }
  }
});

```

```

},
/* Первичный контроль корректности данных договора; сервис повторяет все проверки. */
validate() {
  const errors = {};
  const form = this.form;
  if (!form.clientId) errors.clientId = REQUIRED;
  if (!form.productId) errors.productId = REQUIRED;
  if (!form.number) errors.number = REQUIRED;
  else if (!/^K-\d{6}$/.test(form.number)) errors.number = 'Формат номера договора: K-000001';
  const amount = String(form.amount).trim().replace(',', '.');
  if (!amount) errors.amount = REQUIRED;
  else if (!/^\d{1,12}(\.\d{1,2})?$/ .test(amount)) errors.amount = 'Денежная сумма: только цифры и не более 2 знаков после запятой';
  else if (this.product && (Number(amount) < this.product.minAmount || Number(amount) > this.product.maxAmount))
    errors.amount = 'Сумма кредита по программе: от ' + Bank.money(this.product.minAmount) + ' до ' + Bank.money(this.product.maxAmount) + ' ' + this.product.currency;
  const term = String(form.termMonths).trim();
  if (!term) errors.termMonths = REQUIRED;
  else if (!/^\d{1,3}$/.test(term)) errors.termMonths = 'Срок – целое число месяцев';
  else if (this.product && (Number(term) < this.product.minTermMonths || Number(term) > this.product.maxTermMonths))
    errors.termMonths = 'Срок по программе: от ' + this.product.minTermMonths + ' до ' + this.product.maxTermMonths + ' мес.';
  return errors;
},
checkForm() {
  this.errors = this.validate();
  this.formError = Object.keys(this.errors).length ? 'Форма заполнена с ошибками – исправьте отмеченные поля' : '';
  return !this.formError;
},
async calculate() {
  this.preview = null;
  if (!this.checkForm()) return;
  try {
    this.preview = await Bank.api('POST', '/api/credits/schedule',
      { productId: this.form.productId, amount: this.amount(), termMonths: Number(this.form.termMonths) });
  } catch (e) {
    this.errors = e.fields;
    this.formError = e.message;
  }
},
async save() {
  if (!this.checkForm()) return;
  this.saving = true;
  try {
    const issued = await Bank.api('POST', '/api/credits', {
      number: this.form.number,
      productId: this.form.productId,
      currency: this.form.currency,
      clientId: this.form.clientId,
      amount: this.amount(),
      rate: this.product.rate,
      termMonths: Number(this.form.termMonths),
      startDate: this.meta.bankDate,
      endDate: this.endDate,
    });
    await this.reload();
    await this.openDetails(issued.contract.id);
    this.envelope = issued.card;
    this.notice = 'Договор ' + issued.contract.number + ' заключён, кредит зачислен на счёт ' + issued.contract.mainAccount;
  } catch (e) {
    this.errors = e.fields;
    this.formError = e.message;
  } finally {
    this.saving = false;
  }
},
async openDetails(id) {
  this.notice = '';
  this.error = '';
  this.envelope = null;
  try {
    this.details = await Bank.api('GET', '/api/credits/' + id);
    this.cashAmount = '';
    this.view = 'details';
  } catch (e) {
    this.error = e.message;
  }
},
async cashOut() {
  const contract = this.details.contract;
  const amount = String(this.cashAmount).trim().replace(',', '.');
  this.error = '';
  if (!/^\d{1,12}(\.\d{1,2})?$/ .test(amount) || Number(amount) <= 0) {

```

```

    this.error = 'Укажите сумму выдачи: положительное число, не более 2 знаков после запятой';
    return;
  }
  try {
    await Bank.api('POST', '/api/credits/' + contract.id + '/cash', { amount: Number(amount) });
    await this.openDetails(contract.id);
    this.notice = 'Через кассу выдано ' + Bank.money(amount) + ' ' + contract.currency;
  } catch (e) {
    this.error = e.message;
  }
},
async reissuePin() {
  const contract = this.details.contract;
  try {
    const envelope = await Bank.api('POST', '/api/credits/' + contract.id + '/pin');
    await this.openDetails(contract.id);
    this.envelope = envelope;
  } catch (e) {
    this.error = e.message;
  }
},
async dayClosed(result) {
  this.events = result.events;
  this.meta.bankDate = result.bankDate;
  await this.reload();
  if (this.view === 'details') await this.openDetails(this.details.contract.id);
},
cardNumber(number) {
  return number ? number.replace(/(\d{4})(?=\d)/g, '$1 ') : '';
},
/* Возврат к списку: сообщения карточки договора больше не нужны. */
toList() {
  this.notice = '';
  this.envelope = null;
  this.view = 'list';
},
money: Bank.money,
date: Bank.date,
},
watch: {
  'form.currency'() {
    if (this.product && this.product.currency !== this.form.currency) this.form.productId = null;
  },
},
template: `
<bank-nav active="credits"></bank-nav>
<main>
  <div class="toolbar">
    <div>
      <h1>Кредитные договоры</h1>
      <r class="subtitle">Кредиты физических лиц: заключение договора, график платежей, погашение</r>
    </div>
    <bank-day @changed="dayClosed" @failed="e => error = e.message"></bank-day>
  </div>

  <div v-if="error" id="error" class="alert error">{{ error }}</div>
  <div v-if="notice" id="notice" class="alert success">{{ notice }}</div>
  <div v-if="envelope" id="envelope" class="alert info">
    <b>ПИН-конверт.</b> Карта <span class="mono" id="card-number">{{ cardNumber(envelope.cardNumber) }}</span>,
    PIN-код <b id="card-pin" class="mono">{{ envelope.pin }}</b>.
    PIN-код показывается один раз и в базе не хранится – передайте его клиенту.
  </div>
  <div v-if="events.length" id="events" class="alert info">
    <b>Протокол закрытия дня</b>
    <div v-for="(event, i) in events.slice(-8)" :key="i">{{ event }}</div>
    <div v-if="events.length > 8" class="muted">... всего записей: {{ events.length }}</div>
  </div>

  <section v-if="view === 'list'">
    <div class="toolbar">
      <span class="muted">Договоров: {{ credits.length }}</span>
      <button id="btn-add" class="primary" @click="openForm">Заключить договор</button>
    </div>
    <div class="card">
      <table id="credits">
        <thead>
          <tr>
            <th>Договор</th><th>Клиент</th><th>Вид кредита</th><th class="num">Сумма</th><th class="num">Ставка</th>
            <th>Начало</th><th>Окончание</th><th class="num">Остаток долга</th><th class="num">Уплачено процентов</th><th></th>
          </tr>
        </thead>
      </table>
    </div>
  </section>
`

```

```

<tbody>
  <tr v-for="c in credits" :key="c.id">
    <td class="mono"><a class="link" @click="openDetails(c.id)">{{ c.number }}</a></td>
    <td>{{ c.clientName }}</td>
    <td>{{ c.productName }}<div class="muted" style="font-size: 12.5px">{{ c.kindTitle }}</div></td>
    <td class="num">{{ money(c.amount) }} {{ c.currency }}</td>
    <td class="num">{{ c.rate }} %</td>
    <td>{{ date(c.startDate) }}</td>
    <td>{{ date(c.endDate) }}</td>
    <td class="num">{{ money(c.debt) }}</td>
    <td class="num">{{ money(c.interestPaid) }}</td>
    <td><span class="badge" :class="c.status === 'ACTIVE' ? 'ok' : 'off'">{{ c.status === 'ACTIVE' ? 'Действует' : '
Порашен' }}</span></td>
  </tr>
</tbody>
</table>
<div v-if="!credits.length" class="empty">Договоров пока нет</div>
</div>
</section>

<section v-if="view === 'form'">
  <h2>Новый кредитный договор</h2>
  <div v-if="formError" id="form-alert" class="alert error">{{ formError }}</div>
  <form class="card" novalidate @submit.prevent="save">
    <div class="form-grid">
      <div class="field wide" :class="{ invalid: errors.clientId }">
        <label for="f-clientId">Клиент <span class="req">*</span></label>
        <select id="f-clientId" v-model="form.clientId">
          <option :value="null">— выберите клиента —</option>
          <option v-for="c in clients" :key="c.id" :value="c.id">
            {{ c.fullName }} (нацпоп {{ c.passportSeries }} {{ c.passportNumber }})
          </option>
        </select>
        <div v-if="errors.clientId" class="error" id="e-clientId">{{ errors.clientId }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.number }">
        <label for="f-number">Номер договора <span class="req">*</span></label>
        <input id="f-number" type="text" v-model="form.number" v-mask="'K-####'" placeholder="K-000001">
        <div v-if="errors.number" class="error" id="e-number">{{ errors.number }}</div>
      </div>

      <div class="field">
        <label for="f-currency">Валюта <span class="req">*</span></label>
        <select id="f-currency" v-model="form.currency">
          <option v-for="c in meta.currencies" :key="c.code" :value="c.code">{{ c.code }} — {{ c.name }}</option>
        </select>
      </div>
      <div class="field wide" :class="{ invalid: errors.productId }">
        <label for="f-productId">Вид кредита <span class="req">*</span></label>
        <select id="f-productId" v-model="form.productId">
          <option :value="null">{{ availableProducts.length ? '— выберите программу —' : '— в этой валюте кредиты не
выдаются —' }}</option>
          <option v-for="p in availableProducts" :key="p.id" :value="p.id">
            «{{ p.name }}» — {{ p.kindTitle.toLowerCase() }} {{ p.rate }} % годовых
          </option>
        </select>
        <div v-if="errors.productId" class="error" id="e-productId">{{ errors.productId }}</div>
        <div v-else-if="product" class="hint">{{ product.description }} Сумма {{ money(product.minAmount) }}—{{ money(
product.maxAmount) }} {{ product.currency }},
        срок {{ product.minTermMonths }}—{{ product.maxTermMonths }} мес.</div>
      </div>

      <div class="field" :class="{ invalid: errors.amount }">
        <label for="f-amount">Сумма кредита, {{ form.currency }} <span class="req">*</span></label>
        <input id="f-amount" type="text" v-model="form.amount" placeholder="0.00">
        <div v-if="errors.amount" class="error" id="e-amount">{{ errors.amount }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.termMonths }">
        <label for="f-termMonths">Срок договора, мес. <span class="req">*</span></label>
        <input id="f-termMonths" type="text" v-model="form.termMonths" v-mask="'###'">
        <div v-if="errors.termMonths" class="error" id="e-termMonths">{{ errors.termMonths }}</div>
      </div>
      <div class="field" :class="{ invalid: errors.rate }">
        <label for="f-rate">Процент по кредиту, % годовых</label>
        <input id="f-rate" type="text" :value="product ? product.rate : ''" readonly>
        <div v-if="errors.rate" class="error" id="e-rate">{{ errors.rate }}</div>
      </div>

      <div class="field" :class="{ invalid: errors.startDate }">
        <label for="f-startDate">Дата начала</label>
        <input id="f-startDate" type="text" :value="date(meta.bankDate)" readonly>
        <div v-if="errors.startDate" class="error" id="e-startDate">{{ errors.startDate }}</div>
      </div>
    </div>
  </form>
</section>

```

```

</div>
<div class="field" :class="{ invalid: errors.endDate }">
  <label for="f-endDate">Дата окончания</label>
  <input id="f-endDate" type="text" :value="date(endDate)" readonly>
  <div v-if="errors.endDate" class="error" id="e-endDate">{{ errors.endDate }}</div>
</div>
</div>
<div class="form-actions" style="margin-top: 18px">
  <button id="btn-save" type="submit" class="primary" :disabled="saving">Заключить договор</button>
  <button id="btn-calc" type="button" @click="calculate">Рассчитать график</button>
  <button id="btn-cancel" type="button" @click="view = 'list'">Отмена</button>
</div>
</form>
<div v-if="preview" class="card">
  <h2>График платежей (предварительный расчёт)</h2>
  <schedule-table :schedule="preview" :currency="form.currency"></schedule-table>
</div>
</section>

<section v-if="view === 'details' && details">
  <div class="toolbar">
    <h2>Договор {{ details.contract.number }} · {{ details.contract.clientName }}</h2>
    <button id="btn-back" @click="toList">К списку</button>
  </div>
  <div class="card details">
    <div><div class="label">Вид кредита</div><div class="value">«{{ details.contract.productName }}»</div><div class="muted">
    <div>{{ details.contract.kindTitle }}</div></div>
    <div><div class="label">Сумма кредита</div><div class="value">{{ money(details.contract.amount) }} {{ details.contract.
    currency }}</div></div>
    <div><div class="label">Ставка</div><div class="value">{{ details.contract.rate }} % годовых</div></div>
    <div><div class="label">Статус</div><div class="value">{{ details.contract.status === 'ACTIVE' ? 'Действует' : 'Погашен
    ' + date(details.contract.closedOn) }}</div></div>
    <div><div class="label">Срок</div><div class="value">{{ details.contract.termMonths }} мес.</div></div>
    <div><div class="label">Период</div><div class="value">{{ date(details.contract.startDate) }} – {{ date(details.contract
    .endDate) }}</div></div>
    <div><div class="label">Остаток долга</div><div class="value">{{ money(details.contract.debt) }} {{ details.contract.
    currency }}</div></div>
    <div><div class="label">Уплачено процентов</div><div class="value">{{ money(details.contract.interestPaid) }} {{ details
    .contract.currency }}</div></div>
  </div>

  <div class="card">
    <h2>Карта и выдача наличных</h2>
    <div class="details" style="align-items: end">
      <div><div class="label">Карта к кредитному счёту</div>
      <div class="value mono">{{ cardNumber(details.contract.cardNumber) }}</div>
      <span class="badge" :class="details.contract.cardBlocked ? 'bad' : 'ok'">{{ details.contract.cardBlocked ? '
      Заблокирована' : 'Активна' }}</span></div>
      <div><div class="label">Доступно на счёте</div><div class="value" id="available">{{ mainAccount ? money(mainAccount.
      balance) : '-' }} {{ details.contract.currency }}</div></div>
      <div class="field"><label for="cash-amount">Выдать через кассу, {{ details.contract.currency }}</label>
      <input id="cash-amount" type="text" v-model="cashAmount" placeholder="0.00"></div>
      <div style="display: flex; gap: 8px">
        <button id="btn-cash" class="primary" @click="cashOut">Выдать</button>
        <button id="btn-pin" @click="reissuePin">Перевыпустить PIN</button>
      </div>
    </div>
  </div>

  <div class="card">
    <h2>График платежей</h2>
    <schedule-table :schedule="details.schedule" :currency="details.contract.currency"></schedule-table>
  </div>

  <div class="card">
    <h2>Счета договора</h2>
    <table id="contract-accounts">
      <thead><tr><th>Номер счёта</th><th>Код</th><th>Назначение</th><th>Активность</th><th class="num">Дебет</th><th class="
      num">Кредит</th><th class="num">Сальдо</th></tr></thead>
      <tbody>
        <tr v-for="a in details.accounts" :key="a.number">
          <td class="mono">{{ a.number }}</td>
          <td class="mono">{{ a.chartCode }}</td>
          <td>{{ a.chartName }}<div class="muted" style="font-size: 12.5px">{{ a.name }}</div></td>
          <td>{{ a.activityTitle }}</td>
          <td class="num">{{ money(a.debit) }}</td>
          <td class="num">{{ money(a.credit) }}</td>
          <td class="num"><b>{{ money(a.balance) }}</b></td>
        </tr>
      </tbody>
    </table>
  </div>

```

```

        <div class="card">
            <h2>Проводки по договору</h2>
            <table id="contract-operations">
                <thead><tr><th>Дата</th><th>Операция</th><th>Счёт</th><th class="num">Дебет</th><th class="num">Кредит</th></tr></thead>
                <tbody>
                    <template v-for="op in details.operations" :key="op.id">
                        <tr v-for="(entry, i) in op.entries" :key="op.id + '-' + i">
                            <td>{{ i === 0 ? date(op.bankDate) : '' }}</td>
                            <td>{{ i === 0 ? op.description : '' }}</td>
                            <td class="mono">{{ entry.account }} <span class="muted">{{ entry.chartCode }}</span></td>
                            <td class="num">{{ entry.side === 'DEBIT' ? money(entry.amount) : '' }}</td>
                            <td class="num">{{ entry.side === 'CREDIT' ? money(entry.amount) : '' }}</td>
                        </tr>
                    </template>
                </tbody>
            </table>
        </div>
    </section>
</main>',
    });

app.component('bank-nav', Bank.navComponent);
app.component('bank-day', Bank.dayComponent);
app.component('schedule-table', scheduleTable);
app.directive('mask', Bank.maskDirective);
app.mount('#app');

```

Листинг А.46 – credit-service/src/test/java/by/bsuir/bank/credit/AtmProtocolIT.java

```

package by.bsuir.bank.credit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OperationInfo;
import by.bsuir.bank.credit.atm.DepositApi;
import by.bsuir.bank.credit.atm.DepositApi.DepositInfo;
import by.bsuir.bank.credit.dto.CreditIssued;
import by.bsuir.bank.credit.dto.CreditRequest;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import by.bsuir.bank.credit.repository.MobilePaymentRepository;
import by.bsuir.bank.credit.service.CreditService;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;
import org.springframework.web.client.ResourceAccessException;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

import static by.bsuir.bank.common.ledger.InMemoryLedger.START;
import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

```

```

/** Интеграционные тесты банковской стороны протокола «банк — банкомат». */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import(TestLedgerConfig.class)
class AtmProtocolIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @Autowired
    private CreditService credits;
    @Autowired
    private CreditContractRepository contracts;
    @Autowired
    private CardRepository cards;
    @Autowired
    private MobilePaymentRepository payments;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;
    @MockitoBean
    private DepositApi deposits;

    private CreditIssued credit;
    private String card;
    private String pin;

    @BeforeEach
    void issueCreditWithCard() {
        payments.deleteAll();
        cards.deleteAll();
        contracts.deleteAll();
        ledger.reset();
        when(clients.get(anyLong())).thenAnswer(call ->
            new ClientInfo(call.getArgument(0), "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
        credit = credits.open(new CreditRequest("K-000001", 1L, "BYN", 1L, new BigDecimal("1000.00"),
            new BigDecimal("17.65"), 6, START, START.plusMonths(6)));
        card = credit.card().cardNumber();
        pin = credit.card().pin();
    }

    @Test
    @DisplayName("Авторизация: верный PIN-код принимается")
    void authorize() throws Exception {
        send("AUTHORIZE")
            .andExpect(jsonPath("$.status").value("OK"))
            .andExpect(jsonPath("$.data.holder").value("Иванов Иван Иванович"));
    }

    @Test
    @DisplayName("Три неверных PIN-кода подряд блокируют карту; перевыпуск PIN-кода снимает блокировку")
    void wrongPinThreeTimes() throws Exception {
        send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.code").value("WRONG_PIN"))
            .andExpect(jsonPath("$.data.attemptsLeft").value(2));
        send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(1));
        send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_BLOCKED"));
        send("AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_BLOCKED")); // верный PIN уже не помогает

        String response = mvc.perform(post("/api/credits/" + credit.contract().id() + "/pin"))
            .andExpect(status().isOk()).andReturn().getResponse().getContentAsString();
        pin = json.readTree(response).get("pin").asText();
        send("AUTHORIZE").andExpect(jsonPath("$.status").value("OK"));
    }

    @Test
    @DisplayName("Счётчик неверных попыток сбрасывается после верного PIN-кода")
    void attemptsAreResetAfterSuccess() throws Exception {
        send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(2));
        send("AUTHORIZE").andExpect(jsonPath("$.status").value("OK"));
        send(card, wrongPin(), "AUTHORIZE").andExpect(jsonPath("$.data.attemptsLeft").value(2));
    }

    @Test
    @DisplayName("Неизвестная карта и неполная транзакция отклоняются")
    void unknownCardAndIncompleteTransaction() throws Exception {
        send("9112380000009993", pin, "AUTHORIZE").andExpect(jsonPath("$.code").value("CARD_NOT_FOUND"));
        transaction(List.of(item("CARD", card), item("OPERATION", "BALANCE")))
            .andExpect(jsonPath("$.code").value("BAD_REQUEST"));
        send("TRANSFER").andExpect(jsonPath("$.code").value("UNKNOWN_OPERATION"));
        mvc.perform(post("/api/atm/transactions").contentType(MediaType.APPLICATION_JSON).content("{\"items\": []}"))

```

```

        .andExpect(status().isBadRequest());
    }

    @Test
    @DisplayName("Просмотр остатка кредитного счёта")
    void balance() throws Exception {
        send("BALANCE")
            .andExpect(jsonPath("$.status").value("OK"))
            .andExpect(jsonPath("$.data.balance").value(1000.00))
            .andExpect(jsonPath("$.data.currency").value("BYN"))
            .andExpect(jsonPath("$.data.account").value(credit.contract().mainAccount()))
            .andExpect(jsonPath("$.data.bankDate").value("2026-10-01"));
    }

    @Test
    @DisplayName("Снятие наличных: проводки через кассу, остаток уменьшается")
    void withdraw() throws Exception {
        send("WITHDRAW", item("AMOUNT", "300"))
            .andExpect(jsonPath("$.status").value("OK"))
            .andExpect(jsonPath("$.message").value("Заберите деньги"))
            .andExpect(jsonPath("$.data.amount").value(300))
            .andExpect(jsonPath("$.data.balance").value(700.00));

        String cash = ledger.systemAccount(LedgerApi.CASH, "BYN").number();
        assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("700.00"));
        assertThat(ledger.debit(cash).isEqualByComparingTo("300.00"));
        assertThat(ledger.credit(cash).isEqualByComparingTo("300.00"));
        assertThat(ledger.operations("K-000001").extracting(OperationInfo::description)
            .containsExactly("Выделение кредита банком", "Перевод кредита в кассу", "Получение кредита через кассу"));
    }

    @Test
    @DisplayName("Снятие суммы больше остатка — сообщение об ошибке, счёт не меняется")
    void withdrawMoreThanAvailable() throws Exception {
        send("WITHDRAW", item("AMOUNT", "1001"))
            .andExpect(jsonPath("$.status").value("ERROR"))
            .andExpect(jsonPath("$.code").value("INSUFFICIENT_FUNDS"));

        assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
        // отказ не ломает последующие операции с той же картой
        send("WITHDRAW", item("AMOUNT", "1000")).andExpect(jsonPath("$.status").value("OK"));
    }

    @ParameterizedTest(name = "сумма «{0}»")
    @ValueSource(strings = {"0", "-5", "abc", "10.5", "", "1e3", "12345678901"})
    @DisplayName("Сумма снятия — целое положительное число")
    void withdrawAmountIsValidated(String amount) throws Exception {
        send("WITHDRAW", item("AMOUNT", amount)).andExpect(jsonPath("$.code").value("INVALID_AMOUNT"));

        assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
    }

    @Test
    @DisplayName("Платёж за мобильную связь: деньги уходят на расчётный счёт оператора")
    void payment() throws Exception {
        send("PAYMENT", item("OPERATOR", "MTS"), item("PHONE", "0297654321"), item("AMOUNT", "25.50"))
            .andExpect(jsonPath("$.status").value("OK"))
            .andExpect(jsonPath("$.data.operator").value("MTC"))
            .andExpect(jsonPath("$.data.phone").value("0297654321"))
            .andExpect(jsonPath("$.data.balance").value(974.50))
            .andExpect(jsonPath("$.data.reference").isString());

        AccountInfo operator = ledger.accounts("OPERATOR-MTS-BYN").get(0);
        assertThat(operator.chartCode()).isEqualTo("3012");
        assertThat(operator.balance()).isEqualByComparingTo("25.50");
        assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("974.50"));
        assertThat(payments.findAll().singleElement().satisfies(p -> {
            assertThat(p.getPhone()).isEqualTo("0297654321");
            assertThat(p.getAmount()).isEqualByComparingTo("25.50");
        }));

        // второй платёж тому же оператору идёт на тот же счёт
        send("PAYMENT", item("OPERATOR", "MTS"), item("PHONE", "0331112233"), item("AMOUNT", "10"))
            .andExpect(jsonPath("$.status").value("OK"));
        assertThat(ledger.accounts("OPERATOR-MTS-BYN").hasSize(1));
        assertThat(ledger.balance(operator.number()).isEqualByComparingTo("35.50"));
    }

    @ParameterizedTest(name = "{0} / {1} / {2} -> {3}")
    @CsvSource({
        "BEELINE, 0297654321, 10, UNKNOWN_OPERATOR",
        "A1, 297654321, 10, INVALID_PHONE",
    })

```

```

        "A1, 0177654321, 10, INVALID_PHONE",
        "A1, 02976543210, 10, INVALID_PHONE",
        "A1, 0297654321, 0, INVALID_AMOUNT",
        "A1, 0297654321, 10.555, INVALID_AMOUNT",
        "A1, 0297654321, 1000.01, INSUFFICIENT_FUNDS",
    })
    @DisplayName("Платёж проверяется: оператор, номер из 10 цифр, сумма, достаточность средств")
    void paymentIsValidated(String operator, String phone, String amount, String code) throws Exception {
        send("PAYMENT", item("OPERATOR", operator), item("PHONE", phone), item("AMOUNT", amount))
            .andExpect(jsonPath("$.status").value("ERROR"))
            .andExpect(jsonPath("$.code").value(code));

        assertThat(ledger.balance(credit.contract().mainAccount()).isEqualByComparingTo("1000.00"));
        assertThat(payments.count()).isZero();
    }

    @Test
    @DisplayName("Остаток депозитных счетов запрашивается у сервиса депозитов")
    void depositBalance() throws Exception {
        when(deposits.byClient(1L)).thenReturn(List.of(
            new DepositInfo("Д-000001", "Свободное накопление", "BYN", new BigDecimal("1500.00"), "ACTIVE"),
            new DepositInfo("Д-000002", "Online-решение", "BYN", new BigDecimal("900.00"), "CLOSED")));

        send("DEPOSIT_BALANCE")
            .andExpect(jsonPath("$.status").value("OK"))
            .andExpect(jsonPath("$.data.deposits", hasSize(1)))
            .andExpect(jsonPath("$.data.deposits[0].number").value("Д-000001"))
            .andExpect(jsonPath("$.data.deposits[0].amount").value(1500.00));
    }

    @Test
    @DisplayName("Сервис депозитов недоступен — банкомат получает отказ, а не ошибку сервера")
    void depositServiceIsDown() throws Exception {
        when(deposits.byClient(1L)).thenThrow(new ResourceAccessException("Connection refused"));

        send("DEPOSIT_BALANCE")
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.code").value("SERVICE_UNAVAILABLE"));
    }

    @Test
    @DisplayName("Справочник операторов связи")
    void operators() throws Exception {
        mvc.perform(get("/api/atm/operators")).andExpect(jsonPath("$", hasSize(3)));
    }

    // ----- сборка транзакции банкомата -----

    private ResultActions send(String operation, Map<String, String>... extra) throws Exception {
        return send(card, pin, operation, extra);
    }

    @SafeVarargs
    private ResultActions send(String cardNumber, String pinCode, String operation, Map<String, String>... extra) throws Exception {
        {
            List<Map<String, String>> items = new ArrayList<>(List.of(
                item("CARD", cardNumber), item("PIN", pinCode), item("OPERATION", operation)));
            items.addAll(List.of(extra));
            return transaction(items);
        }

        private ResultActions transaction(List<Map<String, String>> items) throws Exception {
            return mvc.perform(post("/api/atm/transactions").contentType(MediaType.APPLICATION_JSON)
                .content(json.writeValueAsString(Map.of("items", items))))
                .andExpect(status().isOk());
        }

        private static Map<String, String> item(String name, String value) {
            return Map.of("name", name, "value", value);
        }

        private String wrongPin() {
            return "0000".equals(pin) ? "1111" : "0000";
        }
    }
}

```

Листинг А.47 — credit-service/src/test/java/by/bsuir/bank/credit/CardNumbersTest.java

```
package by.bsuir.bank.credit;
```

```

import by.bsuir.bank.credit.domain.Card;
import by.bsuir.bank.credit.domain.CardNumbers;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты номера карты (алгоритм Луна) и счётчика неверных PIN-кодов. */
class CardNumbersTest {

    @ParameterizedTest
    @ValueSource(strings = {"4539148803436467", "4111111111111111", "5500000000000004"})
    void acceptsKnownValidNumbers(String number) {
        assertThat(CardNumbers.isValid(number)).isTrue();
    }

    @ParameterizedTest
    @ValueSource(strings = {"4539148803436468", "4111111111111111", "4111111111111111", "4111-1111-1111-1111", ""})
    void rejectsWrongNumbers(String number) {
        assertThat(CardNumbers.isValid(number)).isFalse();
    }

    @Test
    void generatedNumbersAreValidAndUnique() {
        String first = CardNumbers.generate(1);
        String second = CardNumbers.generate(2);

        assertThat(first).hasSize(16).startsWith("911238").isNotEqualTo(second);
        assertThat(CardNumbers.isValid(first)).isTrue();
        assertThat(CardNumbers.isValid(second)).isTrue();
        assertThat(CardNumbers.isValid(null)).isFalse();
    }

    @Test
    void cardIsBlockedAfterThirdWrongPin() {
        Card card = new Card();

        assertThat(card.registerFailedAttempt()).isEqualTo(2);
        assertThat(card.registerFailedAttempt()).isEqualTo(1);
        assertThat(card.isBlocked()).isFalse();
        assertThat(card.registerFailedAttempt()).isZero();
        assertThat(card.isBlocked()).isTrue();

        card.resetAttempts();
        assertThat(card.isBlocked()).isFalse();
        assertThat(card.getFailedAttempts()).isZero();
    }
}

```

Листинг А.48 – credit-service/src/test/java/by/bsuir/bank/credit/CreditApiIT.java

```

package by.bsuir.bank.credit;

import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.credit.atm.DepositApi;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

```

```

import static org.assertj.core.api.Assertions.assertThat;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/** Интеграционные тесты REST сервиса кредитов: контроль данных договора, график платежей, выдача через кассу. */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@AutoConfigureMockMvc
@Import(TestLedgeConfig.class)
class CreditApiIT {

    @Autowired
    private MockMvc mvc;
    @Autowired
    private ObjectMapper json;
    @Autowired
    private CreditContractRepository contracts;
    @Autowired
    private CardRepository cards;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;
    @MockitoBean
    private DepositApi deposits;

    @BeforeEach
    void cleanState() {
        cards.deleteAll();
        contracts.deleteAll();
        ledger.reset();
        ClientInfo client = new ClientInfo(1L, "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4");
        when(clients.get(1L)).thenReturn(client);
        when(clients.list()).thenReturn(List.of(client));
    }

    @Test
    @DisplayName("Справочники формы: виды кредита, валюты, следующий номер договора")
    void references() throws Exception {
        mvc.perform(get("/api/products")).andExpect(jsonPath("$", hasSize(2)))
            .andExpect(jsonPath("$[0].kindTitle").value("Аннуитетные платежи"))
            .andExpect(jsonPath("$[1].kind").value("INTEREST_ONLY"));
        mvc.perform(get("/api/meta"))
            .andExpect(jsonPath("$.bankDate").value("2026-10-01"))
            .andExpect(jsonPath("$.nextNumber").value("K-000001"));
    }

    @Test
    @DisplayName("График платежей рассчитывается до заключения договора")
    void schedulePreview() throws Exception {
        mvc.perform(post("/api/credits/schedule").contentType(MediaType.APPLICATION_JSON)
            .content("{\"productId\": 1, \"amount\": 10000, \"termMonths\": 6}"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.rows", hasSize(6)))
            .andExpect(jsonPath("$.rows[0].total").value(1753.51))
            .andExpect(jsonPath("$.totalInterest").value(521.06));
        assertThat(contracts.count()).isZero();
    }

    @Test
    @DisplayName("Договор заключается: возвращаются счета, карта и PIN-конверт")
    void createsContract() throws Exception {
        create(valid()).andExpect(status().isCreated())
            .andExpect(jsonPath("$.contract.number").value("K-000001"))
            .andExpect(jsonPath("$.contract.debt").value(10000))
            .andExpect(jsonPath("$.card.cardNumber").isString())
            .andExpect(jsonPath("$.card.pin").isString());

        Long id = contracts.findAll().get(0).getId();
        mvc.perform(get("/api/credits/" + id))
            .andExpect(jsonPath("$.accounts", hasSize(2)))
            .andExpect(jsonPath("$.schedule.rows", hasSize(6)))
            .andExpect(jsonPath("$.operations", hasSize(1)));
        mvc.perform(get("/api/credits")).andExpect(jsonPath("$", hasSize(1)))
            .andExpect(jsonPath("$[0].cardBlocked").value(false));
        mvc.perform(get("/api/meta")).andExpect(jsonPath("$.nextNumber").value("K-000002"));
    }
}

```

```

@ParameterizedTest(name = "без поля {0}")
@ValueSource(strings = {"number", "productId", "currency", "clientId", "amount", "rate", "termMonths", "startDate", "endDate"
})
@DisplayName("Все поля договора обязательны")
void requiredFields(String field) throws Exception {
    Map<String, Object> request = valid();
    request.remove(field);

    create(request).andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + field).value("Обязательное поле"));
    assertThat(contracts.count()).isZero();
}

@ParameterizedTest(name = "{0} = {1}")
@CsvSource({
    "number, Д-000001",
    "amount, 499.99",
    "amount, 50000.01",
    "rate, 10.00",
    "termMonths, 5",
    "termMonths, 61",
    "startDate, 2026-09-30",
    "endDate, 2027-05-01",
    "currency, USD",
})
@DisplayName("Условия договора сверяются с кредитной программой и банковским днём")
void termsAreChecked(String field, String value) throws Exception {
    Map<String, Object> request = valid();
    request.put(field, value);

    create(request).andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.fields." + ("currency".equals(field) ? "productId" : field)).exists());
    assertThat(contracts.count()).isZero();
    assertThat(ledger.journal()).isEmpty();
}

@Test
@DisplayName("Номер договора уникален")
void duplicateNumber() throws Exception {
    create(valid()).andExpect(status().isCreated());

    create(valid()).andExpect(status().isConflict()).andExpect(jsonPath("$.fields.number").exists());
    assertThat(contracts.count()).isEqualTo(1);
}

@Test
@DisplayName("Выдача через кассу: сумма проверяется, превышение остатка отклоняется")
void cash() throws Exception {
    create(valid()).andExpect(status().isCreated());
    String url = "/api/credits/" + contracts.findAll().get(0).getId() + "/cash";

    mvc.perform(post(url).contentType(MediaType.APPLICATION_JSON).content("{\"amount\": 2500.50}"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.balance").value(7499.50));
    mvc.perform(post(url).contentType(MediaType.APPLICATION_JSON).content("{\"amount\": -1}"))
        .andExpect(status().isBadRequest());
    mvc.perform(post(url).contentType(MediaType.APPLICATION_JSON).content("{\"amount\": 7499.51}"))
        .andExpect(status().isConflict())
        .andExpect(jsonPath("$.code").value("INSUFFICIENT_FUNDS"));
}

private Map<String, Object> valid() {
    Map<String, Object> request = new LinkedHashMap<>();
    request.put("number", "K-000001");
    request.put("productId", 1);
    request.put("currency", "BYN");
    request.put("clientId", 1);
    request.put("amount", 10000);
    request.put("rate", 17.65);
    request.put("termMonths", 6);
    request.put("startDate", "2026-10-01");
    request.put("endDate", "2027-04-01");
    return request;
}

private ResultActions create(Map<String, Object> request) throws Exception {
    return mvc.perform(post("/api/credits").contentType(MediaType.APPLICATION_JSON)
        .content(json.writeValueAsString(request)));
}
}

```

Листинг А.49 –

credit-service/src/test/java/by/bsuir/bank/credit/CreditFlowIT.java

```
package by.bsuir.bank.credit;

import by.bsuir.bank.common.api.BankException;
import by.bsuir.bank.common.client.ClientApi;
import by.bsuir.bank.common.client.ClientInfo;
import by.bsuir.bank.common.ledger.AccountInfo;
import by.bsuir.bank.common.ledger.InMemoryLedger;
import by.bsuir.bank.common.ledger.LedgerApi;
import by.bsuir.bank.common.ledger.OperationInfo;
import by.bsuir.bank.credit.atm.DepositApi;
import by.bsuir.bank.credit.dto.CreditDetails;
import by.bsuir.bank.credit.dto.CreditIssued;
import by.bsuir.bank.credit.dto.CreditRequest;
import by.bsuir.bank.credit.dto.CreditView;
import by.bsuir.bank.credit.dto.ScheduleRow;
import by.bsuir.bank.credit.repository.CardRepository;
import by.bsuir.bank.credit.repository.CreditContractRepository;
import by.bsuir.bank.credit.service.DayCloseProcessor;
import by.bsuir.bank.credit.service.CreditService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.context.annotation.Import;
import org.springframework.test.context.bean.override.mockito.MockitoBean;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static by.bsuir.bank.common.ledger.InMemoryLedger.CAPITAL;
import static by.bsuir.bank.common.ledger.InMemoryLedger.START;
import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.when;

/**
 * Интеграционные тесты кредитного процесса: сервис, база H2 и главная книга в памяти.
 * Обороты счетов сверяются со схемой проводок «Кредитная программа» из задания.
 */
@SpringBootTest(properties = "spring.datasource.url=jdbc:h2:mem:${random.uuid};DB_CLOSE_DELAY=-1")
@Import(TestLedgerConfig.class)
class CreditFlowIT {

    private static final long ANNUITY = 1;           // «R-деньги mix», 17,65 %
    private static final long INTEREST_ONLY = 2;     // «R-Онлайн», 17,65 %

    @Autowired
    private CreditService credits;
    @Autowired
    private DayCloseProcessor dayClose;
    @Autowired
    private CreditContractRepository contracts;
    @Autowired
    private CardRepository cards;
    @Autowired
    private InMemoryLedger ledger;
    @MockitoBean
    private ClientApi clients;
    @MockitoBean
    private DepositApi deposits;

    private String cash;
    private String fund;

    @BeforeEach
    void cleanState() {
        cards.deleteAll();
        contracts.deleteAll();
        ledger.reset();
        cash = ledger.systemAccount(LedgerApi.CASH, "BYN").number();
        fund = ledger.systemAccount(LedgerApi.FUND, "BYN").number();
        when(clients.get(anyLong())).thenAnswer(call ->
            new ClientInfo(call.getArgument(0), "Иванов Иван Иванович", "MP", "7654321", "3170590A077PB4"));
    }

    @Test
```

```

@DisplayName("Заключение договора: два активных счёта, выделение кредита из фонда, график и карта")
void openingContract() {
    CreditIssued issued = open("K-000001", ANNUITY, "10000.00", 6);
    CreditView credit = issued.contract();

    List<AccountInfo> accounts = ledger.accounts("K-000001");
    assertThat(accounts).extracting(AccountInfo::chartCode).containsExactly("2400", "2470");
    assertThat(accounts).allSatisfy(a -> assertThat(a.activity()).isEqualTo("ACTIVE"));
    assertThat(ledger.operations("K-000001").extracting(OperationInfo::description)
        .containsExactly("Выделение кредита банком");

    // фонд (пассивный): дебет -10000; текущий счёт (активный): дебет +10000
    assertThat(ledger.balance(fund)).isEqualByComparingTo(CAPITAL.subtract(new BigDecimal("10000.00")));
    assertTurnover(credit.mainAccount(), "10000.00", "0.00", "10000.00");

    assertThat(credits.details(credit.id()).schedule().rows().hasSize(6);
    assertThat(credit.debt()).isEqualByComparingTo("10000.00");
    assertThat(issued.card().cardNumber()).hasSize(16).isEqualTo(credit.cardNumber());
    assertThat(issued.card().pin()).matches("\\d{4}");
    assertThat(cards.findByNumber(credit.cardNumber()).orElseThrow().getPinHash())
        .startsWith("$2").doesNotContain(issued.card().pin());
}

@Test
@DisplayName("Выдача через кассу: с текущего счёта в кассу и из кассы клиенту")
void cashOut() {
    CreditView credit = open("K-000001", ANNUITY, "10000.00", 6).contract();

    AccountInfo after = credits.cashOut(credit.id(), new BigDecimal("4000.00"));

    assertThat(after.balance()).isEqualByComparingTo("6000.00");
    assertTurnover(credit.mainAccount(), "10000.00", "4000.00", "6000.00");
    assertTurnover(cash, "4000.00", "4000.00", "0.00");
    assertThat(ledger.operations("K-000001").extracting(OperationInfo::description)
        .contains("Перевод кредита в кассу", "Получение кредита через кассу");

    assertThatThrownBy(() -> credits.cashOut(credit.id(), new BigDecimal("6000.01")))
        .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getCode()).isEqualTo("INSUFFICIENT_FUNDS"));
    assertThat(ledger.balance(credit.mainAccount()).isEqualByComparingTo("6000.00");
}

@Test
@DisplayName("Аннуитет: в дату платежа проводятся проценты и часть долга, остальные дни счета не меняются")
void annuityPaymentOnDueDate() {
    CreditView credit = open("K-000001", ANNUITY, "10000.00", 6).contract();
    credits.cashOut(credit.id(), new BigDecimal("10000.00"));

    assertThat(closeDays(30)).isEmpty(); // 31.10 – платежей ещё нет
    List<String> events = closeDays(1); // 01.11 – первый платёж
    assertThat(events).containsExactly("01.11.2026: K-000001 – платёж №1: проценты 147.08, основной долг 1606.43 BYN");

    // процентный счёт: дебет +147,08 (перевод из кассы) и кредит -147,08 (начисление банком)
    assertTurnover(credit.interestAccount(), "147.08", "147.08", "0.00");
    // текущий счёт: +10000 выдача, -10000 снятие, +1606,43 погашение из кассы, -1606,43 возврат в фонд
    assertTurnover(credit.mainAccount(), "11606.43", "11606.43", "0.00");
    // касса: транзит выданного кредита и принятого платежа 1753,51
    assertTurnover(cash, "11753.51", "11753.51", "0.00");
    // фонд: -10000 выдача, +147,08 проценты, +1606,43 возврат долга
    assertThat(ledger.balance(fund)).isEqualByComparingTo(CAPITAL.subtract(new BigDecimal("8246.49")));

    CreditDetails details = credits.details(credit.id());
    assertThat(details.contract().debt()).isEqualByComparingTo("8393.57");
    assertThat(details.contract().interestPaid()).isEqualByComparingTo("147.08");
    assertThat(details.schedule().rows().get(0).paidOn()).isEqualTo(LocalDate.of(2026, 11, 1));
    assertThat(details.schedule().rows().get(1).paidOn()).isNull();
}

@Test
@DisplayName("После последнего платежа договор закрывается, банк получает сумму кредита и проценты")
void contractIsClosedAfterLastPayment() {
    CreditView credit = open("K-000001", ANNUITY, "10000.00", 6).contract();
    credits.cashOut(credit.id(), new BigDecimal("10000.00"));

    List<String> events = closeDays(182); // до 01.04.2027 включительно

    assertThat(events).hasSize(7).last().isEqualTo("01.04.2027: K-000001 – кредит погашен, договор закрыт");
    CreditView closed = credits.details(credit.id()).contract();
    assertThat(closed.status()).isEqualTo("CLOSED");
    assertThat(closed.debt()).isEqualByComparingTo("0");
    assertThat(closed.interestPaid()).isEqualByComparingTo("521.06");
    assertThat(ledger.balance(fund)).isEqualByComparingTo(CAPITAL.add(new BigDecimal("521.06")));
    assertThat(ledger.balance(credit.mainAccount()).isEqualByComparingTo("0");
}

```

```

        assertThat(ledger.balance(credit.interestAccount())).isEqualToComparingTo("0");
        assertThat(ledger.operations("K-000001")).extracting(OperationInfo::description).contains("Окончание кредита");
        assertThat(closeDays(40)).isEmpty();
    }

    @Test
    @DisplayName("Кредит с ежемесячной уплатой процентов: долг возвращается последним платежом")
    void interestOnlyCredit() {
        CreditView credit = open("K-000002", INTEREST_ONLY, "6000.00", 4).contract();

        closeDays(92); // три платежа: только проценты
        assertThat(credits.details(credit.id()).contract().debt()).isEqualToComparingTo("6000.00");
        assertThat(credits.details(credit.id()).contract().interestPaid()).isEqualToComparingTo("264.75");
        assertThat(ledger.operations("K-000002")).extracting(OperationInfo::description)
            .doesNotContain("Погашение долга за кредит из кассы");

        closeDays(31); // 01.02.2027 — проценты и весь долг
        CreditView closed = credits.details(credit.id()).contract();
        assertThat(closed.status()).isEqualTo("CLOSED");
        assertThat(closed.interestPaid()).isEqualToComparingTo("353.00");
        assertThat(ledger.balance(fund)).isEqualToComparingTo(CAPITAL.add(new BigDecimal("353.00")));
        // деньги клиент не снимал — выданная сумма осталась на его текущем счёте
        assertThat(ledger.balance(credit.mainAccount())).isEqualToComparingTo("6000.00");
    }

    @Test
    @DisplayName("Повторная обработка дня не проводит платёж второй раз, пропущенные дни навёрстываются")
    void processingIsIdempotentAndCatchesUp() {
        CreditView credit = open("K-000001", ANNUITY, "10000.00", 6).contract();

        assertThat(dayClose.process(START.plusMonths(2))).hasSize(2); // сразу два просроченных платежа
        assertThat(dayClose.process(START.plusMonths(2))).isEmpty();

        assertThat(credits.details(credit.id()).contract().debt()).isEqualToComparingTo("6763.52");
        assertThat(ledger.debit(credit.interestAccount())).isEqualToComparingTo("270.54");
    }

    @Test
    @DisplayName("Сумма сверх лимита программы и заём больше фонда банка отклоняются")
    void limits() {
        assertThatThrownBy(() -> open("K-000001", ANNUITY, "50000.01", 6))
            .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getFields()).containsKey("amount"));
        assertThatThrownBy(() -> open("K-000001", INTEREST_ONLY, "6000.00", 37))
            .isInstanceOfSatisfying(BankException.class, e -> assertThat(e.getFields()).containsKey("termMonths"));
        assertThat(contracts.count()).isZero();
        assertThat(ledger.journal()).isEmpty();
    }

    private CreditIssued open(String number, long productId, String amount, int months) {
        return credits.open(new CreditRequest(number, productId, "BYN", 1L, new BigDecimal(amount),
            new BigDecimal("17.65"), months, START, START.plusMonths(months)));
    }

    /** Закрытие дней подряд — то, что делает сервис «Счета», вызывая участников. */
    private List<String> closeDays(int days) {
        List<String> events = new java.util.ArrayList<>();
        for (int i = 0; i < days; i++) {
            LocalDate next = ledger.bankDay().date().plusDays(1);
            events.addAll(dayClose.process(next));
            ledger.setDate(next);
        }
        return events;
    }

    private void assertTurnover(String account, String debit, String credit, String balance) {
        assertThat(ledger.debit(account)).as("дебет %s", account).isEqualToComparingTo(debit);
        assertThat(ledger.credit(account)).as("кредит %s", account).isEqualToComparingTo(credit);
        assertThat(ledger.balance(account)).as("сальдо %s", account).isEqualToComparingTo(balance);
    }
}

```

Листинг A.50 — credit-service/src/test/java/by/bsuir/bank/credit/ScheduleCalculatorTest.java

```

package by.bsuir.bank.credit;

import by.bsuir.bank.credit.domain.CreditKind;
import by.bsuir.bank.credit.dto.ScheduleRow;
import by.bsuir.bank.credit.dto.ScheduleView;
import by.bsuir.bank.credit.service.ScheduleCalculator;

```

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

/** Модульные тесты расчёта графиков платежей. */
class ScheduleCalculatorTest {

    private static final LocalDate START = LocalDate.of(2026, 10, 1);

    @ParameterizedTest(name = "{0} под {1} % на {2} мес. — платёж {3}")
    @CsvSource({
        "10000.00, 17.65, 6, 1753.51",
        "10000.00, 17.65, 12, 915.14",
        "50000.00, 17.65, 60, 1260.17",
        "1200.00, 0.00, 12, 100.00",
    })
    void annuityPaymentFormula(String amount, String rate, int months, String payment) {
        assertThat(ScheduleCalculator.annuityPayment(new BigDecimal(amount), new BigDecimal(rate), months))
            .isEqualToComparingTo(payment);
    }

    @Test
    void annuityScheduleRepaysWholeDebtWithEqualPayments() {
        List<ScheduleRow> rows = build(CreditKind.ANNUITY, "10000.00", 6);

        assertThat(rows).hasSize(6);
        assertThat(rows).extracting(ScheduleRow::dueDate).containsExactly(
            LocalDate.of(2026, 11, 1), LocalDate.of(2026, 12, 1), LocalDate.of(2027, 1, 1),
            LocalDate.of(2027, 2, 1), LocalDate.of(2027, 3, 1), LocalDate.of(2027, 4, 1));
        assertThat(rows.get(0).interest()).isEqualToComparingTo("147.08"); // 10000 * 17,65 / 1200
        assertThat(rows.get(0).principal()).isEqualToComparingTo("1606.43");
        assertThat(rows.get(0).balanceAfter()).isEqualToComparingTo("8393.57");
        assertThat(rows.subList(0, 5).allSatisfy(row -> assertThat(row.total()).isEqualToComparingTo("1753.51")));
        assertThat(rows.get(5).balanceAfter()).isEqualToComparingTo("0");
        assertThat(sum(rows, true)).isEqualToComparingTo("10000.00");

        // проценты убывают, доля основного долга растёт
        for (int i = 1; i < rows.size(); i++) {
            assertThat(rows.get(i).interest()).isLessThan(rows.get(i - 1).interest());
            assertThat(rows.get(i).principal()).isGreaterThan(rows.get(i - 1).principal());
        }
    }

    @Test
    void lastAnnuityPaymentAbsorbsRoundingError() {
        List<ScheduleRow> rows = build(CreditKind.ANNUITY, "9999.99", 36);

        assertThat(sum(rows, true)).isEqualToComparingTo("9999.99");
        assertThat(rows.get(35).balanceAfter()).isEqualToComparingTo("0");
        assertThat(rows.get(35).total().subtract(rows.get(0).total()).abs()).isLessThan(new BigDecimal("1.00"));
    }

    @Test
    void interestOnlySchedulePaysDebtAtTheEnd() {
        List<ScheduleRow> rows = build(CreditKind.INTEREST_ONLY, "6000.00", 4);

        assertThat(rows).hasSize(4);
        assertThat(rows).allSatisfy(row -> assertThat(row.interest()).isEqualToComparingTo("88.25")); // 6000 * 17,65 / 1200
        assertThat(rows.subList(0, 3).allSatisfy(row -> {
            assertThat(row.principal()).isEqualToComparingTo("0");
            assertThat(row.balanceAfter()).isEqualToComparingTo("6000.00");
        }));
        assertThat(rows.get(3).principal()).isEqualToComparingTo("6000.00");
        assertThat(rows.get(3).total()).isEqualToComparingTo("6088.25");
        assertThat(rows.get(3).balanceAfter()).isEqualToComparingTo("0");
    }

    @Test
    void totalsShowOverpayment() {
        ScheduleView view = ScheduleView.of(build(CreditKind.ANNUITY, "10000.00", 6));

        assertThat(view.totalInterest()).isEqualToComparingTo("521.06");
        assertThat(view.totalPayment()).isEqualToComparingTo("10521.06");
    }

    private static List<ScheduleRow> build(CreditKind kind, String amount, int months) {

```

```

        return ScheduleCalculator.build(kind, new BigDecimal(amount), new BigDecimal("17.65"), months, START);
    }

    private static BigDecimal sum(List<ScheduleRow> rows, boolean principal) {
        return rows.stream().map(principal ? ScheduleRow::principal : ScheduleRow::interest)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}

```

Листинг А.51 – credit-service/src/test/java/by/bsuir/bank/credit/TestLedgerConfig.java

```

package by.bsuir.bank.credit;

import by.bsuir.bank.common.ledger.InMemoryLedger;
import org.springframework.boot.test.context.TestConfiguration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Primary;

/** В тестах вместо REST-клиента сервиса «Счета» подставляется главная книга в памяти. */
@TestConfiguration
public class TestLedgerConfig {

    @Bean
    @Primary
    public InMemoryLedger inMemoryLedger() {
        return new InMemoryLedger();
    }
}

```